

Optimized Implementation of the Elliptic Curve Factorization Method on a Highly Parallelized Hardware Cluster

Ralf Zimmermann

November 2009

Diploma Thesis



TECHNISCHE UNIVERSITÄT BRAUNSCHWEIG

Institut für Theoretische Informatik

Supervisor: Prof. Dr. Dietmar Wätjen



RUHR-UNIVERSITY BOCHUM

Chair for Embedded Security

Supervisor: Prof. Dr.-Ing. Christof Paar

Advisor: Dr.-Ing. Tim Güneysu

Set on November 19, 2009 at 10:30.

Abstract

In this thesis we present a highly efficient, parallel implementation of the elliptic curve method (ECM) on reconfigurable hardware in order to support the general number field sieve algorithm (GNFS).

The GNFS is the most effective algorithm to factor large integers. It can be used to attack the RSA cryptosystem by factoring the composite RSA modulus $N = p \cdot q$. During the sieving step, a large amount of mid-sized integers have to be factored. We can use the ECM to increase the speed of this step.

To provide fast modular arithmetic units, we exploit the digital signal processing elements on a Virtex-4 FPGA and increase the speed of the underlying operations compared to other FPGA implementations. We present an optimized implementation on a Virtex-4 SX35 FPGA, running 24 parallel ECM computations of phase 1 and phase 2 at 200 MHz. We propose an architecture, which can be adapted to factor 66 to 236 bit integers. The necessary hardware changes are negligible, as only the instruction sequence has to be recomputed.

Our results show an improvement by more than 20% compared to all previous publications on FPGAs (which implement or estimate the second phase) both on throughput as well as cost-effectiveness. Our comparison with a recent implementation of the ECM on graphic cards indicate that even though FPGAs do not have the computational power of modern GPUs, FPGAs are more power-effective.

Erklärung/Statement

Hiermit versichere ich, dass ich meine Diplomarbeit selbst verfaßt und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie Zitate kenntlich gemacht habe.

I hereby declare that the work presented in this thesis is my own work and that to the best of my knowledge it is original except where indicated by references to other authors.

Braunschweig, den November 19, 2009

Ralf Zimmermann

Contents

1. Introduction	1
2. Hardware Background	5
2.1. General-purpose and Application-specific processors	5
2.2. Introduction to Field Programmable Gate Arrays	6
2.2.1. BlockRAM	8
2.2.2. Digital Signal Processing	9
2.3. COPACOBANA - A reconfigurable Clustersystem	11
3. Mathematical Background	13
3.1. Elementary Number Theory	13
3.2. Modular Arithmetics	16
3.3. Algebraic Structures	19
3.4. Elliptic Curves	20
3.5. Different forms of elliptic curves	25
4. Elliptic Curve Method	27
4.1. Pollard's $p - 1$ method	27
4.2. Transformation to Elliptic Curves	28
4.3. Stage 1	29
4.4. Stage 2	30
5. Related Work	33
6. Implementation	39
6.1. System-Level View	39
6.2. Component-Level view	41
6.3. Low-Level View	44
6.3.1. Workspace Memory	46
6.3.2. Instruction ROM	48
6.3.3. Modular Arithmetic Units	51
6.4. Elliptic Curve Method	59
6.4.1. Phase 1	60
6.4.2. Phase 2	61

7. Results and Comparison	73
7.1. Results for the Arithmetic Components	73
7.2. Results for the ECM System	75
7.3. Comparison	76
8. Conclusion	83
A. Bibliography	85
B. List of Figures	89
C. List of Tables	91
D. List of Algorithms	93

1. Introduction

In the modern world the need for security and authentication methods is ubiquitous. These methods are used in embedded environments and are part of our day-to-day life, i. e., PayTV, credit cards, wireless communication, ePass or RFID security. But not only private and financial data needs security: manipulation in more complex processes like car-to-car communication or city brake systems might have a life-threatening outcome.

Cryptographic algorithms and protocols are used to prevent misuse of services, to protect data and to ensure data integrity, authenticity and confidentiality. Cryptography allows us for example to protect data from others who do not know a certain secret, or to verify if a message was tempered on the way from sender to recipient.

Cryptography is directly linked with cryptanalysis. Cryptanalysts try to attack algorithms, find security weaknesses or prove an algorithm secure. The crucial part is the definition of a secure algorithm. Schneier states “an algorithm is *unconditionally secure* if, no matter how much ciphertext a cryptanalyst has, there is not enough information to recover the plaintext. [...] only a one-time pad is unbreakable given infinite resources. All other cryptosystems are breakable in a ciphertext-only attack, simply by trying every possible key one by one and checking whether the resulting plaintext is meaningful. This is called a *brute-force attack*.”¹

The exhaustive key search is a simple algorithm to break every encryption by trying out all possible keys - given *enough time and resources*. As stated above none of today’s used systems are unconditionally secure - they are only *computationally secure*. This means the algorithm cannot be broken given current and future available resources like memory storage, computational power or time using the best currently-known attack.

In 1976 the public key cryptography was invented by W. Diffie and M. Hellman in [DH76]. The idea is to use a private key K_d and a public key $K_e = f(K_d)$, generated by some function f . Both keys together form a key pair (K_d, K_e) . The public key is used for encryption, while the private key is used for decryption. The security of this system rely on the requirement to f : the knowledge of both K_e and f yields no information about K_d or allows to calculate the inverse function.

We can categorize the public key algorithms by the mathematical problems the security relies on. Today, the most common used systems rely either on the *discrete*

¹see [Sch95] p. 8

logarithm problem or the *integer factorization problem*. In this thesis, we will concentrate on the integer factorization problem as for example needed to break the RSA public key cryptosystem:

Ronald L. Rivest, Adi Shamir and Leonard Adleman presented the RSA public key algorithm in 1977. Their key generation algorithm uses two secret primes p and q of roughly the same size to calculate a modulus $N = p \cdot q$. One chooses a random number $e : 1 < e < \varphi(N) = (p - 1) \cdot (q - 1)$ and calculates the private integer d solving $e \cdot d \equiv 1 \pmod{\varphi(N)}$. The result is the keypair $((N, e), (N, d))$.

To calculate the private key from (N, e) , we need to find the factors p and q of N . Then we can easily compute d and decrypt messages encrypted with (N, e) . There exist a number of factorization algorithms, for example Fermat's factorization method, Lenstra's *elliptic curve method* (ECM) or complex algorithms like the *General Number Field Sieve* (GNFS). As the RSA modulus contains only two prime factors, we can discard algorithms, designed to factor either small numbers or numbers with small prime factors, for this purpose.

To factor RSA numbers (1024 bits and more), we use the GNFS, as it is the most efficient algorithm for factoring large integers. In the sieving step, we need to factor several billion mid-sized, composite numbers with few prime factors. At this point, we can choose special purpose factoring algorithms like the elliptic curve method to further optimize the runtime of the GNFS. This gives us two starting points for high speed implementations in software and hardware. While the GNFS is very complex and a complete hardware implementation seems ineffective compared to software, we can use dedicated hardware to speed up the smaller factorization steps, creating a hybrid software-hardware factorization system.

In this thesis, we present an optimized implementation of the elliptic curve method in reconfigurable hardware. We chose to work on Xilinx Virtex-4 FPGAs and exploit the powerful dedicated hardware elements. By using the COPACOBANA (Cost Optimized PARallel COde Breaker) FPGA cluster, we can combine up to 128 FPGAs to support the number field sieve algorithm.

This thesis is divided into eight chapters. We start with a description of the hardware background in Chapter 2 and examine the advantages of FPGAs compared to general-purpose processors. The chapter reviews the device specific hardware components and gives a short overview of the COPACOBANA cluster. In Chapter 3, we discuss the mathematical background of this thesis and explain the concepts of the elliptic curve arithmetic. Afterwards, we introduce the elliptic curve method in Chapter 4. As the ECM is based on the Pollard's $p - 1$ algorithm, we start with a short description of this algorithm. Then, we present the original algorithm and the standard continuation of the ECM. In Chapter 5, we review previous work on ECM

implementations on FPGAs: The suggestions and results form the basement for our implementation.

Chapter 6 presents our implementation. We start with an overview of the system architecture and explain the components in detail. Afterwards, we describe our optimized arithmetic unit, reviewing the memory management, instruction handling and the highly optimized modular arithmetic units. The chapter closes with a detailed description of implemented optimizations, compared to the previously suggested algorithms to implement the elliptic curve method.

We discuss our results in the next chapter and start with the modular arithmetic components, followed by the results for the elliptic curve method system and a fair comparison to the related work. The last chapter summarizes the results of the thesis.

2. Hardware Background

In this chapter we discuss the hardware background of this thesis. We start with a short overview on general-purpose and application-specific processors and continue with an introduction to Field Programmable Gate Arrays. As we use a Xilinx Virtex-4 FPGA, we discuss the structure and dedicated hardware resources it provides. The chapter ends with an introduction of the COPACOBANA cluster.

2.1. General-purpose and Application-specific processors

Many applications today run on standard *Central Processing Units* (CPUs). They provide the capability to be programmed in high level languages and include a large instruction set necessary for common applications. If we want to implement a specific algorithm, i. e., fast arithmetic operations, we do not need many of these instructions. In this case, generic processors may not be the most effective platform for this task.

Instead of CPUs, we can use *Application Specific Integrated Circuits* (ASICs). They are very fast, fixed chips we can optimize for one specific task only. As they don't have the overhead of a general-purpose processor and are specialized for the chosen task, ASICs can outperform CPUs. While ASICs have a low material cost, the development and prototyping costs are comparably high. Only large scale production results in a reasonable price per unit.

A different type of IC is the *Field Programmable Gate Array* (FPGA). FPGAs belong to the family of Programmable Logic Devices (PLDs). Depending on the technology used, the FPGA can be reprogrammed, for example to exchange a cryptographic algorithm in an embedded system, in case the original algorithm is no longer considered computationally secure. This also allows rapid prototyping of hardware-based algorithms and a small time to market. While the development costs are low, the (re)configuration overhead leads to lower frequencies compared to ASICs and CPUs.

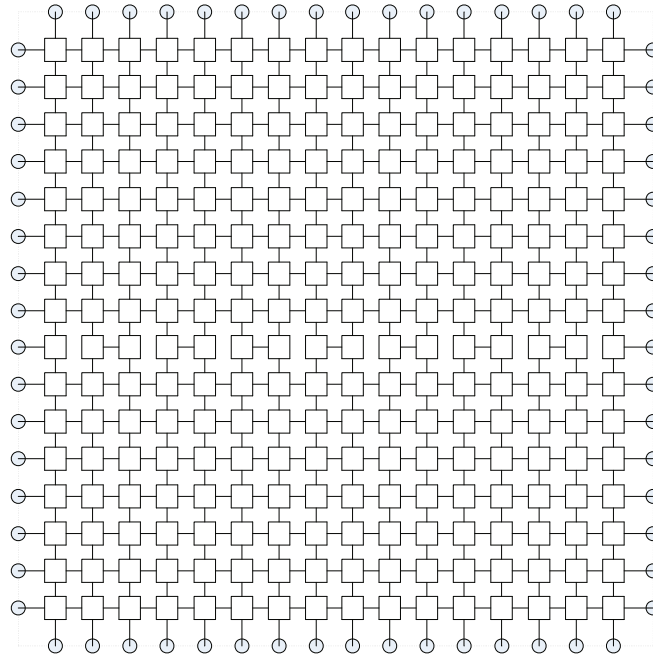


Figure 2.1.: Basic layout of a Field-Programmable Gate Array

2.2. Introduction to Field Programmable Gate Arrays

Figure 2.1 shows the basic layout of an FPGA: it consists of a very large amount of basic logic blocks, called *Configurable Logic Blocks* (CLB). The basic layout of a CLB is some form of *Lookup Table* (LUT) and a *Flipflop* (FF). The detailed layout of the CLB and the functionality varies for different manufacturers and FPGA devices. We can use these CLBs to implement logical functions and connect a series of CLBs. The programming mechanism - and thus the way to configure and connect the blocks - can use different technologies: anti-fuses, SRAM, E(E)PROM / Flash.

Input and Output pins (I/O pins) are placed at the exterior of the chip. Additionally each FPGA contains some mechanism to relay the clock signal and distribute a stable signal throughout the chip. More information on FPGAs and the history of the programmable devices can be found in [KTR08], [RHSDPK06] and [BR96].

In this thesis, we use the Xilinx Virtex-4 SX35 FPGA. As the CLB design and distribution of dedicated hardware blocks may differ compared to other devices, we will describe this FPGA in the rest of this section. The information is based on the different available Xilinx Virtex-4 Documentations².

²see <http://www.xilinx.com/support/documentation/virtex-4.htm> - in particular [Xil08a] and [Xil08b]

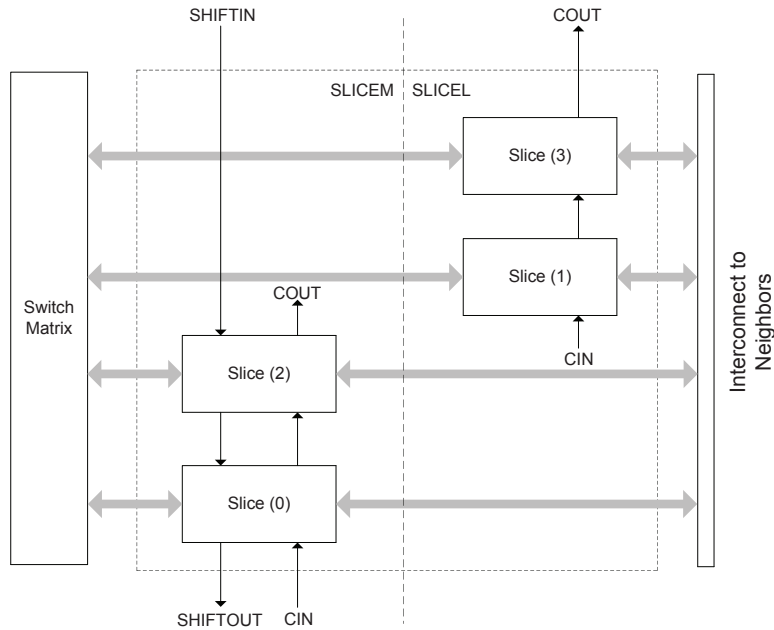


Figure 2.2.: Configurable Logic Block in Virtex-4 FPGAs

CLBs are connected to the general routing matrix and can connect to fast horizontal and vertical long lines. Additionally, the CLBs have an interconnect to their neighbors. A CLB consists of four slices. Figure 2.2 shows this layout. These slices are grouped in pairs and organized in two columns. We can use the SLICEM column as logic, distributed RAM or a shift register. The second column, SLICEL, can implement only logic.

Each slice contains two function generators for Boolean functions. The Virtex-4 family uses 4-input LUTs with independent inputs for each of the two function generators. This means they are capable of implementing any four-input Boolean function. In addition to these LUTs, the Slices contain multiplexers. They can combine up to eight function generators in one CLB, providing an implementation of functions with five to eight inputs. If not used for Boolean functions, the LUTs can store data as distributed RAM, read-only memory or operate as shift register. More detailed information on the possible slice usage can be found in [Xil08a].

In addition to the basic CLBs the Virtex-4 contains dedicated hardware. The important hardware blocks for this thesis are the *BlockRAM* units, the *Xtreme DSP48* cores and the *Digital Clock Manager* (DCM). The DCM uses the input clock and generates a stable output clock of a different frequency; e.g., we use multiple clock domains in this thesis with a cascade of two DCMs .

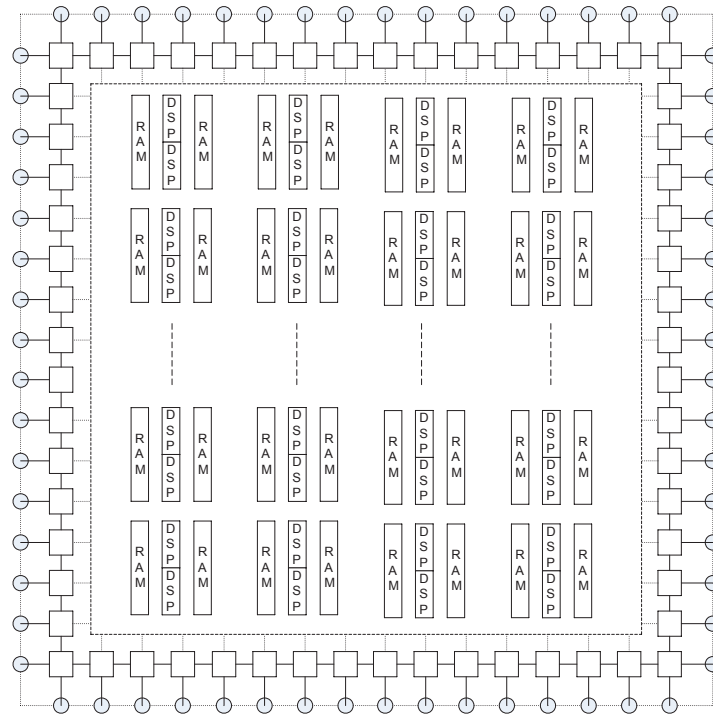


Figure 2.3.: Distribution of dedicated hardware units on Virtex-4 FPGAs

2.2.1. BlockRAM

The Virtex-4 FPGAs contain BlockRAM (BRAM) units. We can use the maximum device frequency - which is 400 MHz using our device - to access the memory blocks, storing up to 18 Kbits of data. These dedicated memories have integrated pipeline registers inside the module, which decrease the critical path in the module and lead to the maximum frequency.

The distribution of the BlockRAMs on the FPGA is outlined in figure 2.3. The units are placed in columns over the FPGA - in case of the XC4VSX35, there are eight columns of 24 BlockRAM units each - surrounded by the CLBs. A total of 192 units is available for use. We can combine adjacent units without external logic and cascade the blocks to large memory storage. Even though the unit is called BlockRAM, we can use it either as *Read Only Memory* (ROM) or as *Random Access Memory* (RAM). We can choose from a set of predefined memory types, such as single port memory, dual port memory (using one port for read and one for write operations) and true dual port memory.

Each BRAM contains two independent ports to access the memory. We can either use the same (synchronous) clock or different, asynchronous clocks for the ports.

Each port consists of an address and data output bus. Depending on the configuration, a read and/or write enable and output reset signal, as well as an input bus are available. We can define the read and write width for each port as one of the following: 1, 2, 4, 9, 18 and 36. Other settings are possible, but result in the usage of more than one BlockRAM unit.

When we use a dual port RAM, we can adjust the write behavior as follows: When in write first mode, the data is simultaneously written into the memory and stored in the output register. Read first on the other hand, first reads the memory at the address into the output register and then writes the new data to the address. In the last setting, the no change mode, the output registers stay unchanged during write operations.

These modes have a major impact on memory control sequences and can lead to different conflicts. While two parallel read operations are always allowed, a simultaneous read and write operation on the same address must follow these rules: Using asynchronous clocks, while one port writes to an address the other must not access the same memory address within a specified clock-to-clock setup time. Using synchronous clocks, both ports may write the same data to the same address. Furthermore is a parallel write/read operation only allowed in read first mode. The behavior and memory content in case of a violation is unpredictable.

Apart from the normal memory storage, we can use the BRAMs in a different way: the Xilinx Core Generator allows us to easily configure the memory as a FIFO buffer. We can use either synchronous or asynchronous clocks and connect optional control signals like (half) full/empty. There are many applications for the BlockRAM units. In this thesis, we use these memory blocks as dual port ROM, true dual port RAM and FIFO buffer, both with synchronous and asynchronous clocks.

2.2.2. Digital Signal Processing

The Virtex-4 was the first Virtex FPGA with dedicated *Digital Signal Processing* (DSP) elements, called XtremeDSPTM. These elements can perform computations used in digital signal processing at maximum device speed. These operations include multiply, multiply and accumulate, three input addition and barrel shifting.

The DSP hardcores are aligned in column like the BlockRAM units. As seen in figure 2.3, two DSP slices are combined into a DSP tile of the size of one BRAM. The XC4VSX35 FPGA contains four columns of 48 DSP cores each and thus 192 DSP cores. Each DSP column is surrounded by two BRAM columns.

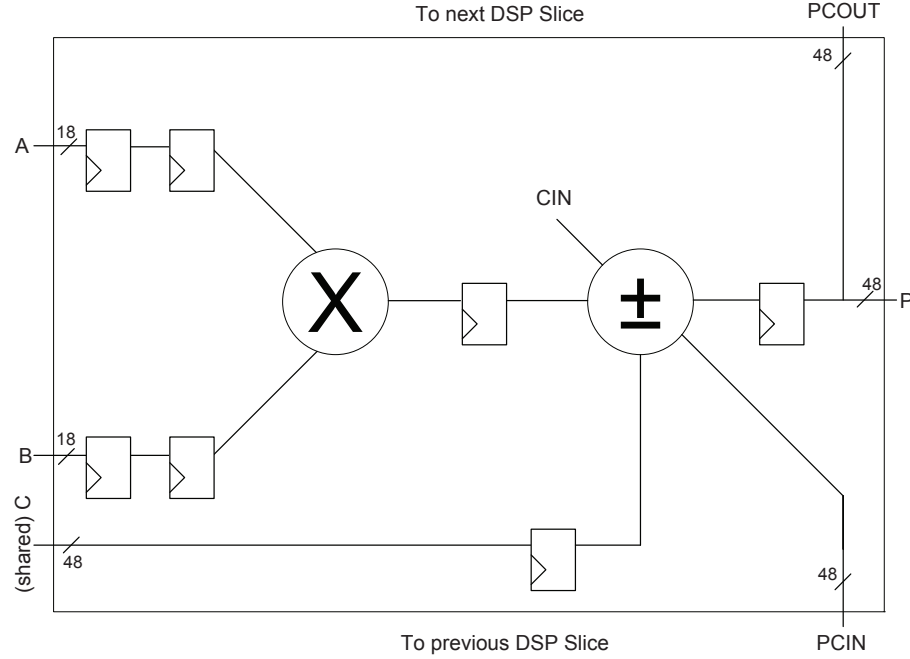


Figure 2.4.: Simplified view of a DSP48 Slice

Figure 2.4 shows a simplified view of a DSP slice with several details left out³, i. e., multiplexers and internal feedback paths. The slice contains two 18 bit inputs A and B. Each can use up to two pipeline registers inside the slide. In addition, A 48 bit shared input C is available: One DSP tile shares this input for both slices. The result, indicated as port P, can use an internal high-speed datapath to connect to the next DSP slice.

The most important feature of the DSP slice for our thesis is the signed 18x18 multiplication of A and B, followed by an 48 bit addition one clock cycle later. We use this mode to build our modular multiplication unit. Also, both arithmetic modules use the signed 48 bit addition. To compute a three-input addition in one clock cycle, the inputs A and B are concatenated (further denoted by $[A : B]$) and used together with either the result P from the previous calculation (using the internal feedback path) or the shared C input.

We remark that we only presented the operation modes used in this thesis⁴. Furthermore, we can change the mode of operation in each clock cycle. We use this feature to implement the latest version of our modular addition unit.

³see [Xil08b], p. 18, for a detailed layout plan of a DSP48 Tile

⁴see [Xil08b], p. 29f for the complete list of operation modes.

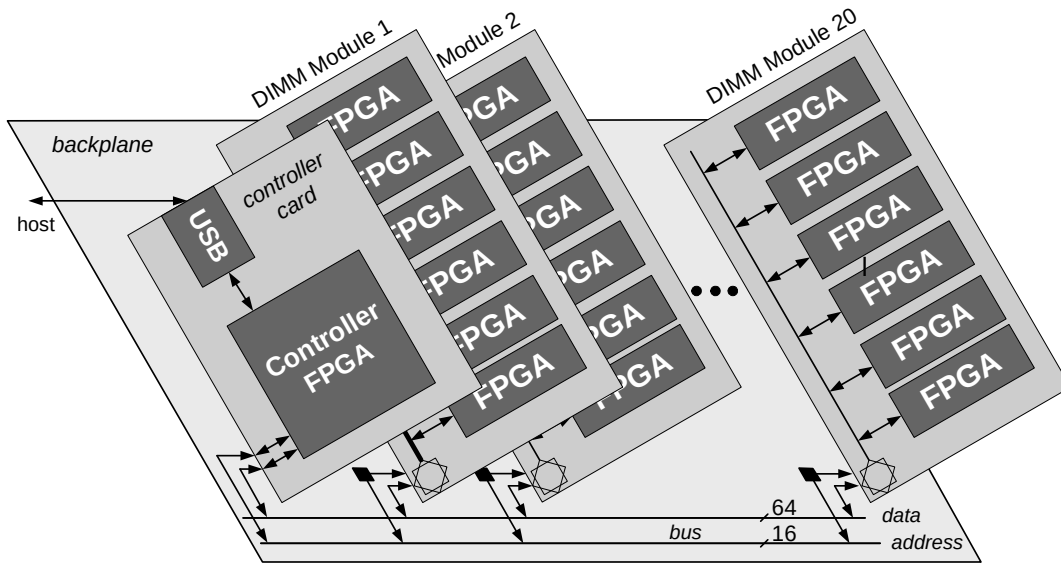


Figure 2.5.: Overview: Layout of the modified COPACOBANA, based on [GPPS08]

2.3. COPACOBANA - A reconfigurable Clustersystem

In 2006, Kumar *et al.* presented a special reconfigurable hardware cluster with low-cost FPGAs in [KPP⁺06]: the Cost-Optimized Parallel Code Breaker - in short COPACOBANA. It is a special designed, highly optimized hardware cluster. The first version used up to 120 Spartan-3 1000 FPGAs. It lacked additional memory or high speed connections, which were not necessary for the implemented cryptographic purpose: The first application of the COPACOBANA was a brute-force attack on the Data Encryption Standard (DES). The cluster tested 400 million keys per second and the exhaustive key search was completed after 9 days.

An enhanced version was suggested in [GPPS08] using Virtex-4 FPGAs using a Gigabit connection. The COPACOBANA revision we use in this thesis has been released in May 2008. Figure 2.5 shows the basic layout. Our configuration uses four modules, each containing eight Virtex-4 SX35 FPGAs with speed grade 10.

We use a host PC to control the machine and connect to an integrated PC (μ ATX) in the 19" case of the COPACOBANA. We transfer data and send control signals via Ethernet. The integrated PC manages the communication to the COPACOBANA and connects using USB to the Cluster. This way, we can manage and access the computation power of the COPACOBANA over a network.

3. Mathematical Background

In this chapter we discuss the mathematical background of this thesis. At first a short summary on modular arithmetics and group theory is given, followed by the definition of elliptic curves and point operation on these curves. The chapter ends by introducing projective coordinates and Montgomery Curves. For more detailed information and proofs, we refer to [Knu97,HMV04,Sti02,Gün09].

Some mathematical set notation will be useful:

- $\mathbb{R}$, the set of real numbers,
- $\mathbb{Z} := \{\dots, -1, 0, 1, \dots\}$, the set of integer numbers,
- $\mathbb{N} := \{0, 1, 2, \dots\}$, the set of nonnegative integer numbers,
- $\mathbb{P}$, the set of prime numbers.

For number sets M containing 0, we define $M^* := M \setminus \{0\}$. In particular, $\mathbb{N}^* = \mathbb{N} \setminus \{0\}$ is the set of natural numbers.

3.1. Elementary Number Theory

In elementary number theory one analyses the basic properties of integers. In connection to the area of cryptography and cryptanalysis the divisability properties and methods as well as factorization are of special importance. At first, we introduce modular arithmetic. We remind that the *floor* of a real number r is given by

$$\lfloor r \rfloor := \max\{z \in \mathbb{Z} \mid z \leq r\}.$$

Definition 3.1.1 (Remainder) For all $a, b \in \mathbb{Z}$, we call

$$a \bmod b := \begin{cases} a & b = 0, \\ a - b \lfloor \frac{a}{b} \rfloor & b \neq 0. \end{cases}$$

the remainder of the integer division of a by b ; b is called the modulus.

Example 3.1.1 $15 \bmod 4 = 15 - 4 \lfloor \frac{15}{4} \rfloor = 15 - 4 \cdot 3 = 3$.

For $n \in \mathbb{N}^*$, $\lfloor \frac{a}{n} \rfloor \leq \frac{a}{n}$ implies $0 \leq a \bmod n \leq n - 1$. For $n \in \mathbb{N}^*$, the set of integers modulo n is denoted by

$$\mathbb{Z}_n := \{0, 1, \dots, n - 1\} = \{a \bmod n \mid a \in \mathbb{Z}\},$$

and its nonzero elements form the set $\mathbb{Z}_n^*$.

Obviously, a is a *divisor* of $b \neq 0$, denoted by $a \mid b$, if and only if $a \bmod b = 0$. Equivalently, we say that b is an *integral multiple* of a .

Definition 3.1.2 (Congruence (modulo m)) For $a, b, m \in \mathbb{Z}$, we call a congruent b modulo m , denoted by $a \equiv b \pmod{m}$, if $a \bmod m = b \bmod m$.

We remark that the definition of the remainder immediately leads to: $a \bmod m = b \bmod m \Rightarrow a = b + km, k \in \mathbb{N}$.

Equivalent to $a \equiv b \pmod{m}$, the difference $a - b$ is an integral multiple of m . Calculations with congruences in modular arithmetic satisfy some easy basic rules:

Theorem 3.1.3 For all $a, a_1, b, b_1, c, d, m \in \mathbb{Z}$,

1. (Reflexivity): $a \equiv a \pmod{m}$,
2. (Symmetry): $a \equiv b \pmod{m}$ if and only if $b \equiv a \pmod{m}$,
3. (Transitivity): if $a \equiv b \pmod{m}$ and $b \equiv c \pmod{m}$ then $a \equiv c \pmod{m}$,
4. if $a \equiv a_1 \pmod{m}$ and $b \equiv b_1 \pmod{m}$ then
 - a) $(a \pm b) \equiv (a_1 \pm b_1) \pmod{m}$,
 - b) $(ab) \equiv a_1 b_1 \pmod{m}$.

Proof: All these properties immediately follow from the definition. Proofs are included to gain some familiarity with the definition.

1. $a = a + xm$ for $x = 0 \in \mathbb{Z}$.
2. If $a = b + xm$ for some $x \in \mathbb{Z}$ then $b = a + (-x)m$ for $-x \in \mathbb{Z}$ and vice versa.
3. If $a = b + xm$ and $b = c + ym$ for some $x, y \in \mathbb{Z}$ then $a = (c + ym) + xm = c + (y + x)m$ for $(y + x) \in \mathbb{Z}$.
4. If $a = a_1 + xm$ and $b = b_1 + ym$ for some $x, y \in \mathbb{Z}$ then
 - a) $(a \pm b) = (a_1 + xm) \pm (b_1 + ym) = (a_1 \pm b_1) + (x \pm y)m$ for $(x \pm y) \in \mathbb{Z}$ and

$$\text{b) } (ab) = (a_1 + xm)(b_1 + ym) = a_1b_1 + a_1ym + b_1xm + xym^2 = a_1b_1 + (a_1y + b_1x + xym)m \text{ for } (a_1y + b_1x + xym) \in \mathbb{Z}. \quad \square$$

The theorem asserts that we may calculate with congruences in the same way as known for numbers. Now, we introduce the greatest common divisor (gcd) and its calculation.

Definition 3.1.4 (gcd) For $a, b \in \mathbb{Z}^*$ the greatest common divisor $d \in \mathbb{N}^*$, denoted by $d = \gcd(a, b)$, is defined by

1. $d \mid a, d \mid b$ and
2. if, for $c \in \mathbb{N}^*$, $c \mid a$ and $c \mid b$ then $c \mid d$.

The calculation of the gcd of two integers is particularly easy if all prime factors of both integers are known. It is well known, that every integer can be factored into prime (powers).

Theorem 3.1.5 (Fundamental Theorem of Arithmetic) Every integer larger than 1 admits a unique prime factorization, i. e., for all $n \in \mathbb{N}$, $n > 1$,

$$n = \prod_i p_i^{e_i}$$

for some prime numbers $p_i \in \mathbb{P}$ and some exponents $e_i \in \mathbb{N}^*$.

There are various algorithms to find the factorization including special-purpose algorithms using the properties of integers (cf. Pollard's $p - 1$ algorithm or the Elliptic Curve Method, cf. chapter 4), as well as general-purpose algorithms like the General Number Field Sieve, where the run-time depends only on the size of the composite number. No algorithm is known for factoring an integer in polynomial time. However, the greatest common divisor can be calculated using the fast Euclidean algorithm without any knowledge about the prime factorization.

Now, we define co-primes and present Fermat's little theorem.

Definition 3.1.6 (co-prime) For two integers a and b , a is called co-prime to b if $\gcd(a, b) = 1$.

For $n \in \mathbb{N}^*$, the set of all integers modulo n which are co-prime to n is denoted by

$$\mathbb{Z}_n^{co} := \{x \in \mathbb{Z}_n^* \mid \gcd(x, n) = 1\}.$$

The Euler totient function (or Euler φ function) is a multiplicative function used to compute the number of elements in multiplicative groups.

Definition 3.1.7 (Euler totient function) For $n \in \mathbb{N}^*$, the Euler totient function is $\varphi(n) := |\mathbb{Z}_n^{co}|$.

Remark 3.1.1 Obviously for $p \in \mathbb{P}$, $\varphi(p) = p - 1$.

The following theorem of Fermat is important for many cryptographic systems like the Pohlig-Hellman exponentiation or the RSA cipher.

Theorem 3.1.8 (Fermat's little theorem) For $a \in \mathbb{Z}^*$ and $p \in \mathbb{P}$,

$$a^{p-1} \equiv 1 \pmod{p}$$

For the proof, we refer to [Knu97], p. 41.

A slightly generalized variant is used in the RSA cryptosystem. Theorem 3.1.8 was generalized by Euler.

Theorem 3.1.9 (Euler's theorem) If $a \in \mathbb{Z}^*$ is co-prime to $n \in \mathbb{N}^*$, then

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

We conclude this section ends with definitions of some *smoothness criteria* for integers relative to their prime factors.

Definition 3.1.10 (B-smooth) For $m, B \in \mathbb{N}^*$, we call m *B-smooth*, if all prime factors of m are limited by B , i. e., for all $p \in \mathbb{P}$ with $p|m$, $p < B$.

Definition 3.1.11 (B-powersmooth) For $m, B \in \mathbb{N}^*$, we call m *B-powersmooth*, if all prime powers dividing m are limited by B , i. e., for all $p \in \mathbb{P}$ and for all $e \in \mathbb{N}^*$ with $p^e|m$, $p^e < B$.

3.2. Modular Arithmetics

In this section we describe the algorithms for modular addition, modular subtraction and modular multiplication used in this thesis.

Algorithm 1 implements modular addition and subtraction. It is straight-forward: after the basic calculation a single reduction step is necessary if the result $z \notin \mathbb{Z}_M$.

Algorithm 1 Modular addition/subtraction

Input: Modulus M , Integer $a, b \in \mathbb{Z}_M$ **Output:** $r = a \pm b \bmod M$ $r \leftarrow a \pm b$ **if** $r \geq M$ **then** {Reduction in Addition} $r \leftarrow r - M$ **else if** $r < 0$ **then** {Reduction in Subtraction} $r \leftarrow r + M$ **end if****return** r

Algorithm 2 Modular multiplication

Input: Modulus M , Integer $a, b \in \mathbb{Z}_M$ **Output:** $r = a \cdot b \bmod M$ $s \leftarrow a \cdot b$ $r \leftarrow s - \lfloor \frac{s}{M} \rfloor M$ **return** r

The inverse operation used in the reduction step is not expensive to implement in software or hardware.

Algorithm 2 implements modular multiplication. In the reduction step, we calculate the remainder of the integer division $\frac{a \cdot b}{M}$. The disadvantage in this classical modular multiplication is this expensive division in the reduction step.

In 1985, Montgomery presented an algorithm which is faster than the naive modular multiplication with a large modulus in [Mon85]: It replaces the division by a simple shift operation which are very efficient in soft- and hardware.

For a given modulus M , one chooses a radix $R > M$, R co-prime to M . For computational efficiency, a configuration of $R = 2^{wn}$ using w bit registers and n words per multi-precision integer is recommendable. An integer $a \in \mathbb{Z}_M$ is converted to Montgomery form A by computing $A := aR \bmod M$.

While the addition of two integers in this Montgomery representation is unchanged ($A + B = aR + bR = (a + b)R$), the multiplication $A \cdot B = aRbR = abR^2$ needs an additional reduction.

The idea of the Montgomery multiplication is to calculate a multiple of M and add it to $X = AB$. Hence the result v is not changed modulo M but additionally is

turned into a multiple of R :

$$\begin{aligned}
 v &= X + fM \\
 &\equiv X + (XM')M \\
 &\equiv X + (-XM^{-1})M \\
 &\equiv 0 \pmod{R}
 \end{aligned}$$

Thus, the division by R is performed without remainder. This is the advantage of the Montgomery multiplication: the division can be implemented using a right-shift of $\log_2 R$ bits, which is very fast compared to multi-precision division. The final reduction step ensures $0 \leq x < M$ because

$$\begin{aligned}
 x &= \frac{X + fM}{R} \\
 &< \frac{RM + RM}{R} \\
 &= 2M
 \end{aligned}$$

The Montgomery Reduction needs the constant $M' = -M^{-1} \pmod{R}$, $0 \leq M' < R$ for the pair (M, R) . Using Algorithm 3, the backward reduction of an integer $X < RM$ to $x := XR^{-1} \pmod{M}$ is done very efficiently by constructing a multiple of R ($v = X + fM$) and simple division by R .

Algorithm 3 Montgomery Reduction $MRed(X)$

Input: Modulus M , radix $R > M$ with $\gcd(R, M) = 1$, $M' = -M^{-1} \pmod{R}$, unsigned integer $X, 0 \leq X < RM$

Output: $x = XR^{-1} \pmod{M}$, $0 \leq x < M$

$f \leftarrow X \cdot M' \pmod{R} \{f \in \mathbb{Z}_R\}$

$v \leftarrow X + f \cdot M$

$x \leftarrow v/R$

if $x \geq M$ **then** {Reduction}

$x \leftarrow x - M$

end if

return x

There are many different implementation options using the Montgomery reduction combined with multi-precision multiplication $X = A \cdot B$. Koç *et al.* compared these in [cKKABSK96]. Several implementations on different hardware platforms including graphic cards and FPGAs are described in [Gün09].

The algorithm implemented in this thesis is the “Modular Multiplication with Quotient Pipelining” by Orup [Oru95]. It is presented in Chapter 6.4 along with its implementation.

3.3. Algebraic Structures

We recall some notation from algebra which will be used in the discussion of operations on elliptic curves.

Definition 3.3.1 (Group) *A group $(G, *)$ consists of a non-empty set G and a binary operation $*$: $G \times G \rightarrow G$ with the following properties:*

1. (Closure): $\forall a, b \in G: a * b \in G,$
2. (Associativity): $\forall a, b, c \in G: a * (b * c) = (a * b) * c,$
3. (Neutral element): $\exists e \in G \quad \forall a \in G: a * e = e * a = a,$
4. (Inverse elements): $\forall a \in G \exists b \in G: a * b = b * a = e.$

The binary operation is not commutative in arbitrary groups.

Definition 3.3.2 (Abelian Group) *A group $(G, *)$ is called Abelian if*

5. (Commutativity): $\forall a, b \in G: a * b = b * a$

One can use the definition of Abelian groups to give a short definition of a field. In general, a field consists in a nonempty set $\mathbb{F}$, for example $\mathbb{F} = \mathbb{R}$, endowed with two binary operations which are usually denoted by $+$ and $\cdot$.

Definition 3.3.3 (Field) *We call a nonempty set $\mathbb{F}$ endowed with an additive, binary operation $+$ and a multiplicative, binary operation $\cdot$ a field, denoted by $(\mathbb{F}, +, \cdot)$, if*

1. $(\mathbb{F}, +)$ is an Abelian group with neutral element 0.
2. $(\mathbb{F} \setminus \{0\}, \cdot)$ is an Abelian group with neutral element 1.
3. (Distributive law): $\forall a, b, c \in \mathbb{F}: (a + b) \cdot c = a \cdot c + b \cdot c$

In the following we will only consider finite fields. While being only a small special subset of the class of general fields, it is of crucial importance for cryptography.

Definition 3.3.4 (Finite Field) *A field $\mathbb{F}$ is called finite, if $|\mathbb{F}| < \infty$.*

For a finite field, the number q of its elements is called the *order* of the field. It is well known that the order q of finite fields is always a power of a prime number, i. e., $q = p^m$ for some $p \in \mathbb{P}$, $m \in \mathbb{N}^*$. Vice versa, up to isomorphism, there is exactly one finite field of order p^m for every prime number $p \in \mathbb{P}$ and every natural number $m \in \mathbb{N}^*$. Some further notions for finite fields are:

Definition 3.3.5 (Finite Fields $\mathbb{F}_q$ of order q) We denote a finite field of order $q = p^m$ for some $p \in \mathbb{P}$, $m \in \mathbb{N}^*$ by $\mathbb{F}_q$. Then

1. p is called the characteristic of $\mathbb{F}_q$
2. for $m = 1$, $\mathbb{F}_q$ is called a prime field,
3. for $m \geq 2$, $\mathbb{F}_q$ is called an extension field,
4. for $q = 2^m$, $\mathbb{F}_q$ is called a binary field.

Prime fields are the main examples used in later sections and chapters of this thesis. The canonical prime field is $\mathbb{F}_p \cong (\mathbb{Z}_p, +, \cdot)$ where the addition and multiplication of the integers in $\mathbb{Z}_p$ are calculated modulo $p \in \mathbb{P}$.

3.4. Elliptic Curves

Elliptic curves have been studied by mathematicians since the middle of the 19th century, but not connected to cryptography for a long time. They were an important part in the proof of Fermat's last theorem (see [Wil95]). Other applications cover number theory, complex analysis and algebraic geometry. In 1984, Hendrik Lenstra invented the elliptic curve method (see Chapter 4). A year later, Victor Miller [Mil86] and Neal Koblitz [Kob87] proposed the elliptic curve cryptography.

We cover the basic arithmetics on elliptic curves in this chapter. For a more detailed introduction and advanced uses, we refer to [HMV04, CFA⁺06]. An elliptic curve over a general field $\mathbb{F}$ is a non-singular cubic curve in two variables.

Definition 3.4.1 (Elliptic Curve) An elliptic curve $\mathcal{E}$ over a general field $\mathbb{F}$ is defined by

$$\mathcal{E} := \{(x, y) \mid y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6\} \quad (3.1)$$

for some coefficients $a_i \in \mathbb{F}$, $i = 1 \dots 6$ with discriminant $\Delta \neq 0$, where

$$\begin{aligned}\Delta &:= -d_2^2 d_8 - 8d_4^3 - 27d_6^2 + 9d_2 d_4 d_6 \\ d_2 &= a_1^2 + 4a_2 \\ d_4 &= 2a_4 + a_1 a_3 \\ d_6 &= a_3^2 + 4a_6 \\ d_8 &= a_1^2 + 4a_2 a_6 - a_1 a_3 a_4 + a_2 a_3^2 - a_4^2\end{aligned}\tag{3.2}$$

As known from the algebra, the discriminant of an algebraic equation relates to the number and form of solutions. The assumption $\Delta \neq 0$ is an important requirement for elliptic curves that implies non-singularity.

For fields $\mathbb{F}$ with the characteristic $p \neq 2, p \neq 3$, the equations (3.1) and (3.2) can be simplified by a change of variables.

Definition 3.4.2 (Elliptic Curve: Weierstrass equation) Let $\mathbb{F}$ be a field with characteristic $p \notin \{2, 3\}$. Then an elliptic curve $\mathcal{E}$ over $\mathbb{F}$ is defined by the Weierstrass equation in

$$\mathcal{E} := \{(x, y) \mid y^2 = x^3 + ax + b\}\tag{3.3}$$

for some coefficients $a, b \in \mathbb{F}$ with $\Delta = -16(4a^3 + 27b^2) \neq 0$.

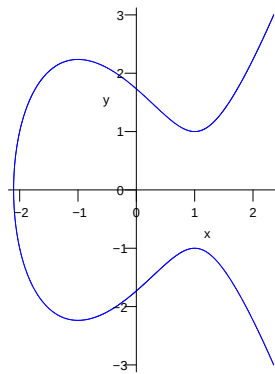


Figure 3.1.: Graph of the elliptic curve $y^2 = x^3 - 3x + 3$ over $\mathbb{R}$

Example 3.4.1 The discriminant of the equation $y^2 = x^3 - 3x + 3$ satisfies $\Delta \neq 0$. Figure 3.1 shows a part of the curve over $\mathbb{R}$.

We will only work with points from an elliptic curve over finite fields.

Definition 3.4.3 ($\mathbb{F}_q$ -rational points) Let $\mathbb{F}_q$ be a finite field with characteristic $p \notin \{2, 3\}$. Then the set of the $\mathbb{F}_q$ -rational points of $\mathcal{E}$ is defined by

$$\mathcal{E}(K) = \{(x, y) \in \mathbb{F}_q \times \mathbb{F}_q \mid x, y \text{ satisfies Equation (3.3)}\} \cup \{\mathcal{O}\}$$

where $\mathcal{O}$ denotes the point of infinity.

We will discuss the meaning and use of the point of infinity when a point addition is constructed. At first, we consider the number of elements of $\mathcal{E}(\mathbb{F}_q)$.

Definition 3.4.4 (Order of the elliptic curve) We call $|\mathcal{E}(\mathbb{F}_q)|$ the order of an elliptic curve $\mathcal{E}$ over the finite field $\mathbb{F}_q$.

A simple but naive way for finding the number of elements is just testing and counting all $x \in \mathbb{F}_q$ for which there exists some $y \in \mathbb{F}_q$ satisfying the equation (3.1). Fortunately, we know some estimate of the number of points for elliptic curves over finite fields.

Theorem 3.4.5 (Hasse) Let $\mathcal{E}$ be an elliptic curve over $\mathbb{F}_q$. Then

$$q + 1 - 2\sqrt{q} \leq |\mathcal{E}(\mathbb{F}_q)| \leq q + 1 + 2\sqrt{q}$$

For large order q of the finite field, theorem 3.4.5 yields the asymptotic estimate $|\mathcal{E}(\mathbb{F}_q)| \approx q$ for the order of elliptic curves. Finding the exact number of points is important for elliptic curve cryptosystems depending on the Elliptic Curve Discrete Logarithm Problem. In 1985, Schoof presented a polynomial time algorithm for elliptic curves over finite fields which has been improved in the Schoof-Elkiens-Atkin algorithm (cf. [Dew98]). In 1999, Satoh presented another method for elliptic curves over binary fields with variants SST and AGM.

In the remaining part of this section, we will introduce a group addition of points on elliptic curves over prime fields $\mathbb{F}_p$ for some prime p . We will start a motivating geometric construction of the addition $+$ in $\mathcal{E}(\mathbb{F}_p)$ in three steps, followed by the algebraic definition. For the construction, the cases $P \neq Q$ (Figure 3.2(a)) and $P = Q$ (Figure 3.2(b)) are handled separately.

1. (Define line G): $G = \begin{cases} \text{through } P \text{ and } Q & \text{if } P \neq Q, \\ \text{tangent line through } P & \text{if } P = Q, \end{cases}$
2. (Define point S of intersection): $S \in G \cap \mathcal{E}(\mathbb{F}_p), S \notin \{P, Q\}$,
3. (Define point R of reflection): R of S by reflection on the x axis.

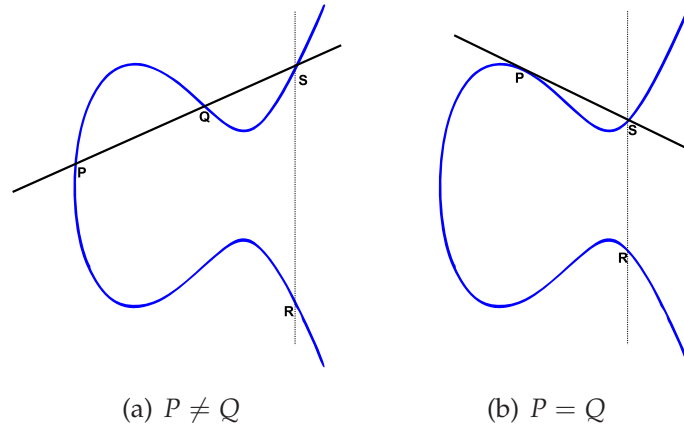


Figure 3.2.: Construction of the point addition: $R = P + Q$, P, Q with $R \in \mathcal{E}(\mathbb{F}_p)$.

If there is no point of intersection between the line and the curve, $S := R := \mathcal{O}$ is the point of infinity.

Besides the geometric motivation, we have to define $(x_3, y_3) = R := P + Q \in \mathcal{E}(\mathbb{F}_p)$, i. e., the addition of two points $P = (x_1, y_1), Q = (x_2, y_2) : P, Q \in \mathcal{E}(\mathbb{F}_p)$, in algebraic terms. At first, we consider the case $P \neq Q$. If $x_1 = x_2$ then $y_1 = -y_2$ which leads to $R := \mathcal{O}$. Otherwise, $x_1 \neq x_2$, and the linear equation of the line is

$$\begin{aligned} y &:= mx + n, \\ m &:= \frac{\Delta y}{\Delta x} = \frac{y_2 - y_1}{x_2 - x_1}, \quad n := y_1 - mx_1. \end{aligned} \tag{3.4}$$

Rewriting the equation of the elliptic curve (3.3) as

$$0 = x^3 + ax + b - y^2$$

and inserting y from the linear equation (3.4) leads to

$$\begin{aligned} 0 &= x^3 + ax + b - (mx + n)^2 \\ &= x^3 - m^2x^2 + (a - 2mn)x + (b - n^2). \end{aligned} \tag{3.5}$$

We already know two zeros x_1 and x_2 of Equation (3.5). Thus, we know that complete polynomial factorization leads to

$$0 = (x - x_1)(x - x_2)(x - x_3) \tag{3.6}$$

and we can find the third zero by comparison of coefficients:

$$x_3 = m^2 - x_1 - x_2, \tag{3.7}$$

$$y_3 = m(x_3 - x_1) + y_1. \tag{3.8}$$

In the case of $P = Q$ the tangent line through P is used in Figure 3.2(a). If $y_1 = 0$, then $G \cap \mathcal{E}(\mathbb{F}_p) = \emptyset$ leads to $R = \mathcal{O}$. Otherwise $y_1 \neq 0$ and the ascent of the tangent is

$$m = \frac{3x_1^2 + a}{2y_1}.$$

Insertion of this formula for m into Equation (3.5) and factoring leads to Equation (3.6) with a double zero $x_1 = x_2$. Comparison of coefficients again leads to Equation (3.7) and (3.8). We summarize point addition in the following definition.

Definition 3.4.6 (Point addition + on $\mathcal{E}(\mathbb{F}_p)$) Let $P = (x_1, y_1), Q = (x_2, y_2) \in \mathcal{E}(\mathbb{F}_p)$ and $-P := (x_1, -y_1)$. Then

$$P + \mathcal{O} := P =: \mathcal{O} + P, \quad P + (-P) = \mathcal{O},$$

and, for $Q \notin \{P, -P\}$, $P + Q := R = (x_3, y_3) \in \mathcal{E}(\mathbb{F}_p)$ with

$$m := \begin{cases} \frac{y_2 - y_1}{x_2 - x_1} & x_1 \neq x_2 \\ \frac{3x_1^2 + a}{2y_1} & x_1 = x_2 \end{cases}$$

$$x_3 := m^2 - x_1 - x_2$$

$$y_3 := m(x_3 - x_1) - y_1$$

Remark 3.4.1 Endowed with the point addition + the rational points of the elliptic curve form an Abelian group $(\mathcal{E}(\mathbb{F}_p), +)$.

We derive scalar multiplication of a natural number $s \in \mathbb{N}$ and a point $P \in \mathcal{E}(\mathbb{F}_p)$ from point addition in the usual way.

Definition 3.4.7 (Scalar multiplication) For $s \in \mathbb{N}, P \in \mathcal{E}(\mathbb{F}_p)$, scalar multiplication is defined by

$$sP = \sum_{n=0}^{s-1} P = \underbrace{P + P + \cdots + P}_{s \text{ times}}$$

Different algorithms for fast scalar multiplication are known (cf. [HMOV04]). Since $\mathbb{F}_p$ is cyclic, there exist a natural number $s \in \mathbb{N}$ such that $sP = \mathcal{O}$. We conclude this chapter with a remark on the order of a point.

Definition 3.4.8 For $P \in \mathcal{E}(\mathbb{F}_p)$, we call $s(P) := \min\{s \in \mathbb{N} \mid sP = \mathcal{O}\}$ the order of P .

Remark 3.4.2 For $P \in \mathcal{E}(\mathbb{F}_p)$,

1. P generates a subgroup $\langle P \rangle$ of order $s(P)$ in $\mathcal{E}(\mathbb{F}_p)$,
2. $s(P) \leq |\mathcal{E}(\mathbb{F}_p)|$.

3.5. Different forms of elliptic curves

There exist different representations and forms of curves, such as projective representation or the Montgomery form. They lead to different implementations and necessary modular arithmetic operations for adding two points. While every elliptic curve can be transformed into Weierstrass form, this is not true for all other forms.

Instead of using the normal elliptic curve (Weierstrass) equation, using a projective form already increases the efficiency to compute point addition.

Definition 3.5.1 (Projective Weierstrass Equation) Let $\mathcal{E}(\mathbb{F}_q)$ be an elliptic curve over a finite field $\mathbb{F}_q$ with $a, b \in \mathbb{F}_q$. The corresponding projective Weierstrass equation is defined by

$$Y^2Z = X^3 + aXZ^2 + bZ^3. \quad (3.9)$$

The projective point $(X : Y : Z)$, $Z \neq 0$ corresponds to $(\frac{X}{Z}, \frac{Y}{Z})$ in affine coordinates. $Z = 0$ leads to the point of infinity $\mathcal{O}$ which corresponds to $(0 : 1 : 0)$.

Montgomery Curves were introduced by Peter Montgomery in [Mon87]. They are derived from Equation (3.3) by change of variables.

Definition 3.5.2 Let $(x, y) \in \mathbb{F}_q^2$ be a point of the Montgomery Curve for $a, b \in \mathbb{F}_q$. Then the Montgomery Curve is defined by the following equation:

$$by^2 = x^3 + ax^2 + x$$

We can also use projective coordinates for Montgomery Curves.

Definition 3.5.3 (Projective Montgomery Equation) Let $(x : y : z) \in \mathbb{F}_q^3$ be a point in projective form of the Montgomery Curve for $a, b \in \mathbb{F}_q$. Then, the corresponding projective equation is:

$$by^2z = x^3 + ax^2z + xz^2$$

with $b(a^2 - 4) \neq 0$.

Remark 3.5.1 We denote Montgomery curves explicitly by $\mathcal{E}_M$.

Montgomery used this form to derive formulas for the addition of two points P and Q and for doubling a point P without using (and computing) the y -coordinate. Using these formulas it is possible to reduce the number of modular multiplications to 6 respectively 5 multiplications (counting squaring as multiplication). The only requirement is that the difference $P - Q$ is already known.

Theorem 3.5.4 *Let $P = (P_x : P_y : P_z)$, $Q = (Q_x : Q_y : Q_z)$ be two points of the Montgomery Curve. Correct addition is possible using the following formulas:*

Addition ($P \neq Q$):

$$\begin{aligned}(P + Q)_x &:= (P - Q)_z((P_x - P_z)(Q_x + Q_z) + (P_x + P_z)(Q_x - Q_z))^2 \\ (P + Q)_z &:= (P - Q)_x((P_x - P_z)(Q_x + Q_z) - (P_x + P_z)(Q_x - Q_z))^2\end{aligned}$$

Doubling ($P = Q$):

$$\begin{aligned}4P_xP_z &:= (P_x + P_z)^2 - (P_x - P_z)^2 \\ a_{24} &:= \frac{a + 2}{4} \\ (2P)_x &:= (P_x + P_z)^2(P_x - P_z)^2 \\ (2P)_z &:= 4P_xP_z((P_x - P_z)^2 + a_{24}(4P_xP_z))\end{aligned}$$

In the context of the elliptic curve method (see Chapter 4) most implementations use Montgomery curves. They have been studied and used for many years and various hardware implementations are known (cf. [PSK⁺05, GKB⁺06]).

4. Elliptic Curve Method

In this chapter, we give an overview of the Elliptic Curve Method (ECM) for factoring integers. First Pollard's $p - 1$ method [Pol74] is introduced, as the main idea of ECM is based on this algorithm and on William's $p + 1$ method [Wil82]. Then, we outline the algorithm of the elliptic curve method and describe the standard extension for phase 2.

4.1. Pollard's $p - 1$ method

Pollard's $p - 1$ -method has been invented in 1974 by John Pollard. It uses Fermat's little theorem in its construction. For factoring a composite number $n \in \mathbb{N}^*$, we consider a prime p with $p|n$. If we choose $a \in \mathbb{Z}$ co-prime to n then, by Theorem 3.1.8,

$$a^{p-1} \equiv 1 \pmod{p}.$$

For $m \in \mathbb{N}$ this equation extends to $a^{m(p-1)} \equiv 1 \pmod{p}$ since $(a^{p-1})^m \equiv 1^m \equiv 1 \pmod{p}$. Substituting $k = m(p - 1)$, we get

$$\begin{aligned} a^{m(p-1)} &= a^k \equiv 1 \pmod{p}, \\ \text{i. e., } a^k - 1 &\equiv 0 \pmod{p}, \\ \text{i. e., } p &|(a^k - 1). \end{aligned}$$

Now, a^k is a multiple of p . If $a^k - 1$ is not divisible by n , a non-trivial divisor of n can be retrieved, since $a^k \not\equiv 1 \pmod{n}$ implies

$$1 < \gcd(a^k - 1, n) < n$$

While this method seems straight-forward, the unknown prime factor p is needed to compute $k = m(p - 1)$. To circumvent this difficulty, $p - 1$ is assumed to decompose into small prime factors. We guess some k as the maximum B_1 -smooth number for a certain bound $B_1 \in \mathbb{N}$ and hope that this k will work:

$$\begin{aligned} k &= \prod_{p_i \in \mathbb{P}, p_i < B_1} p_i^{e_i}, \\ e_i &= \left\lfloor \log_{p_i}(B_1) \right\rfloor \end{aligned} \tag{4.1}$$

If k is indeed a multiple of the unknown $p - 1$, a factor d can be recovered by computing

$$d := \gcd(a^k - 1, n).$$

Unfortunately, d can also be a trivial factor and then the method fails.

4.2. Transformation to Elliptic Curves

The Elliptic Curve Method was proposed in 1985 as a modification of Pollard's $p - 1$ -method. Instead of using the multiplicative group in $\mathbb{Z}_p^*$, H. W. Lenstra suggested the use of the group $(\mathcal{E}(\mathbb{Z}_n), +)$ on elliptic curves [Len87].

Translation of the construction of Pollard's $p - 1$ method motivates a related ECM. We start with a prime factor q of the composite number n and with the elliptic curve $\mathcal{E}(\mathbb{Z}_q)$. If the order of the curve is B_1 -smooth, the construction in Equation (4.1) leads to some multiple k of $q - 1$. In $\mathcal{E}(\mathbb{Z}_q)$, this leads to $Q = kP = \mathcal{O}$ for any point $P \in \mathcal{E}(\mathbb{Z}_q)$. In projective coordinates this implies a division by Q_z and $Q_z \equiv 0 \pmod{q}$. As in the $p - 1$ method, we operate on $\mathcal{E}(\mathbb{Z}_n)$ instead of $\mathcal{E}(\mathbb{Z}_q)$.

We remark that though $(\mathbb{Z}_n, +, \cdot)$ is not a field (as $n \notin \mathbb{P}$), the group operations can be used. In fact, they reduce to arithmetic operations in $\mathbb{Z}/n\mathbb{Z}$. The only critical operation is the inversion $a^{-1} \pmod{n}$ but, if it fails, a non-trivial factor of n is found (unless $a \equiv 0 \pmod{n}$).

Hence $Q = kP$ is computed in $\mathcal{E}(\mathbb{Z}_n)$ hoping to get a point Q with

$$\begin{aligned} Q_z &\not\equiv 0 \pmod{n}, \\ Q_z &\equiv 0 \pmod{q}. \end{aligned}$$

The factor q can be calculated from $q = \gcd(Q_z, n)$.

The main advantage of the elliptic curve method compared to Pollard's $p - 1$ method is that the return of a trivial factor does not lead to a complete failure: When trying to find a prime factor q using an elliptic curve $\mathcal{E}(\mathbb{Z}_q)$ with order g , Hasse's theorem (3.4.5) limits the order g as follows

$$|g - (q + 1)| < 2\sqrt{q}$$

By adjusting the curve parameters, g behaves like a random integer in this interval. In case of failure, a different curve is chosen with a different order. Using Pollard's $p - 1$ method, the order of the multiplicative group is constant and the method cannot be restarted.

There have been many improvements since the proposal of the ECM. A major extension was the introduction of a second phase by Brent in [Bre86] and improving the probability of success. Please note that in literature the terms stage and phase are exchangeable. Montgomery presented further variants of this phase in [Mon87]. Additionally the parametrization of the curve can raise the success rate further. Using Montgomery curves leads to a factor 4 dividing $|\mathcal{E}(\mathbb{Z}_q)|$. Using Suyama's parametrization ensures an additional factor of 3 (see [ZD06]).

The runtime of ECM for finding a factor q given a number n is

$$O(L(q)^{\sqrt{2}+o(1)} \text{Mul}(\log n))$$

with $L(q) = e^{\sqrt{\log q \log \log q}}$ and the complexity of modular multiplication $\text{Mul}(\log n)$. While it mainly depends on the size of q [Bre86], the modular multiplication depends on the size of n . Thus, improvements in the arithmetic and algorithmic performance change different parts in this estimation: "Mathematical and algorithmic improvements influence the $L(q)^{\sqrt{2}+o(1)}$ factor, while arithmetic improvements influence the $M(\log n)$ factor."⁵.

4.3. Stage 1

In the first phase, a factor of the composite number n is found if the order of the curve is B_1 -smooth.

Algorithm 4 Elliptic Curve Method - Phase 1

Input: Composite integer $n = \prod p_i^{e_i}$, $p_i \in \mathbb{P}, e_i \in \mathbb{N}^* : p_i^{e_i} \mid n$, arbitrary curve $\mathcal{E}(\mathbb{Z}_n)$, random point $P \in \mathcal{E}(\mathbb{Z}_n) \neq \mathcal{O}$, smoothness bound $B_1 \in \mathbb{N}$ and maximum B_1 -smooth integer k as in (4.1).

Output: Factor $d \mid n$

$Q \leftarrow kP \{Q = (Q_x : Q_y : Q_z)\}$

$d \leftarrow \gcd(Q_z, n)$

return d

Algorithm 4 shows the necessary steps: After computing a curve and a random point on the curve, the maximum B_1 -smooth number k is calculated. Afterwards the multiplication of P with the scalar k is computed and then a factor is retrieved from z -coordinate.

⁵see [ZD06], Chapter 1.2 - Complexity of ECM

When using Montgomery Curves and the formulas presented in Chapter 3.5, the difference $P - Q$ is needed when adding two points P and Q . Montgomery presented a suitable scalar multiplication method to compute $Q = kP$, denoted as the Montgomery Ladder Algorithm, and described in Algorithm 5.

Algorithm 5 Montgomery Ladder Algorithm

Input: $P_0 = (x_0 : : z_0), k = (k_{n-1}k_{n-2} \dots k_0), \forall i : k_i \in \{0, 1\}, k_{n-1} = 1$

Output: $Q = k \cdot P_0$

$Q \leftarrow P_0, P \leftarrow 2P_0$

for $i := n - 2$ **downto** 0 **do**

if $k_i = 1$ **then**

$Q \leftarrow P + Q, P \leftarrow 2P$

else

$Q \leftarrow 2Q, P \leftarrow P + Q$

end if

end for

The algorithm requires $\log_2 k - 1$ computations of both a point addition and a point doubling. This static runtime is the algorithm's disadvantage: it could be improved by pre-calculating an optimal addition chain. Montgomery designed a heuristic algorithm in [Mon92], called PRAC, to compute these *Lucas* chains.

4.4. Stage 2

The first phase may fail to find a non-trivial factor of the integer n . It may happen that it finds all prime factors and returns n itself. In this case the bound B_1 needs to be adjusted to split the small factors. If, on the other hand, the returned factor is 1, the calculated point $Q = kP$ can be used for further computations.

The main idea is that there exists a prime π in the interval $[B_1, B_2]$ using a certain boundary B_2 which leads to $\pi Q = \mathcal{O} \bmod p$. Then the gcd will again yield the non-trivial factor p of n . As with the bound B_1 in the first phase, B_2 restricts the computational effort and is used to adjust the success probability.

There is no advantage in computing πQ using the first phase's algorithm, as it would simply continue. For simpler explanation, we will describe the continuation using Weierstrass coordinates instead of projective coordinates. Instead of computing each πQ , the prime is represented as $\pi = s + t$ and the multiples $sQ = (S_x, S_y)$ and $tQ = (T_x, T_y)$ are computed. Thus $sQ + tQ = \mathcal{O} \pmod{p}$ implies $S_x \equiv T_x \pmod{p}$. Then, $\gcd(S_x - T_x, n)$ can be used to retrieve the factor.

Algorithm 6 Elliptic Curve Method - Standard continuation

Input: Parameters from ECM Phase 1 plus: smoothness bound B_2 and result $Q = kP$ of Phase 1.

Output: Factor $d \mid n$

$s \leftarrow 1$

for each prime $\pi \in [B_1, B_2]$ **do**

$T \leftarrow \pi Q \{T = (T_x : T_y : T_z)\}$

$s \leftarrow s \cdot T_z$

end for

$d \leftarrow \gcd(s, n)$

return d

This approach exploits the maximal gap between two successive primes. Using an integer $d < B_2$ the values of s and t to reach all primes up to B_2 are:

$$s \in S = \{i \cdot d \mid 0 \leq i \cdot d < B_2\},$$
$$t \in T = \{j \mid 0 < j < d, j \text{ co-prime to } d\}.$$

To further improve this method, we can exploit the fact that

$$\forall j \in \mathbb{N} : (jQ)_x = (-jQ)_x, (jQ)_y = -(-jQ)_y$$

by restricting j in the set T to $j \leq \frac{d}{2}$.

Other extensions and methods to speed up the elliptic curve method such as the fast polynomial arithmetic, fast polynomial multiplication, block windows, the Brent-Suyama's Extension and Montgomery's $d_1 d_2$ Improvement are described in [ZD06]. An algorithm to implement Phase 2 on FPGAs efficiently has been presented in [GKB⁺06] and forms the basis for the optimized implementation in Chapter 6.4.

5. Related Work

In this chapter we present the related work on the topic of implementing the Elliptic Curve Method as a co-factorization unit in hardware.

Pelzl *et al.* published the idea to use the ECM as an acceleration of the general number field sieve 2005 in [PSK⁺05] and provided a hardware implementation to execute ECM phase 1. They came to the conclusion that an implementation of the elliptic curve method on special hardware can drastically decrease the running time of GNFS. In their paper, they chose to factor integers up to 198 bits, using smoothness boundaries of $B_1 = 960$ and $B_2 = 57000$ and a scalar k of 1375 bits as ECM parameters, and elliptic curves in Montgomery form. If not stated otherwise, the succeeding implementations use the same configuration.

Their design used a special arithmetic-logic unit to compute the point operations. To obtain the result $Q = kP$ of the scalar multiplication in phase 1, they used the Montgomery ladder algorithm, computing a Montgomery multiplication in their basic modular arithmetic. Pelzl *et al.* remarked that the speed of the first phase can be improved with a pre-computed, optimal addition chain.

A Virtex-E 2000 FPGA with speed grade 6 was used to implement the ECM system. In addition, an embedded micro-controller managed the control logic.

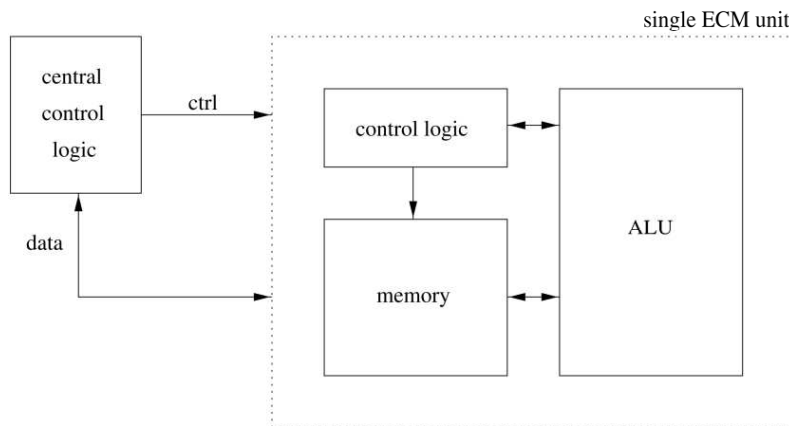


Figure 5.1.: Layout of ECM unit. Taken from [PSK⁺05], p.5

Figure 5.1 shows the proposed layout: Each ECM unit consists of the arithmetic unit, a local memory and control logic. The system is controlled by a central control logic.

While they implemented a working arithmetic unit, they state that “the control logic of phase 1 has not been optimized yet” and “provide estimates for expected timings for phase 1 (and phase 2) based on the timings of the elementary operations”⁶. These estimations were commonly used in the subsequent papers and make comparability less straightforward.

In 2006, Gaj *et al.* presented an improved implementation of the elliptic curve method in [GKB⁺06]. They removed the embedded micro-controller and synthesized the design on both low-cost and high-cost FPGAs: A Virtex-E 2000 with speed grade 6 for comparison with Pelzl *et al.*, a Virtex-II 6000 with speed grade 6, a Spartan-3 5000 with speed grade 5, a Spartan-3E 1600E with speed grade 5 and the largest available Virtex 4 FPGA, the Virtex-4 LX200 with speed grade 11.

Their arithmetic unit uses one adder/subtractor and two multipliers to compute the operations for phase 1 and 2. They came to the conclusion that high-cost FPGAs are unsuited in terms of cost-effectiveness: The low-cost Spartan 3E achieved 380 ECM/s per 100\$ in their optimized design. Comparing their design with Pelzl *et al.* on the Virtex-E showed an improvement factor of 9.3. The timings for the second phase are estimated, supported by simulation.

Gaj *et al.* updated their results in 2008 in the draft [GKB⁺08]. While the results in their previous publication based on a single ECM unit, e.g., the reported maximum frequency, they published the results of the complete ECM system using the maximum number of ECM units: The maximum frequencies slightly decreased and the cycle count for phase 2 increased. Furthermore, they optimized several FPGA implementations and were able to place more parallel ECM units. These results are used for further comparison.

A year later, de Meulenaer *et al.* published an implementation of the ECM in [dMGdDQ07]. Their design exploited the dedicated DSP hardcores of a Virtex-4 FPGA to calculate the modular multiplication. They concentrated their efforts on building a high performance, fully pipelined, unrolled modular multiplication which needs one clock cycle to calculate the result. Instead of using the standard Montgomery multiplication, they implemented a variant by Orup in [Oru95], called “Montgomery Tail Tailoring multiplication” in [Har04]. For this multiplication unit, they needed all available DSP cores to multiply up to “135-bit (SX25), 169-bit (SX35, with 8 additional LUT-based multipliers) and 271-bit (SX55)”⁷, limited by the number of digital signal processing elements on the FPGA.

Their results are based on an implementation on the Virtex-4 SX25 FPGA with speed grade 10, capable of factoring integers up to 135 bits. The design did not target

⁶ [PSK⁺05], p. 9

⁷ [dMGdDQ07], Chapter 4.1

the second phase of the elliptic curve method. They remarked that the expected number of FPGAs necessary to factor a 1024 bit RSA modulus is sufficient large enough to build dedicated high-throughput hardware in the first phase. Compared to a combined solution, this approach increases the bandwidth requirements.

In contrast to all other implementations, they used only one multiplication unit and two addition and subtraction circuits due to the latency and pipelining. Although the design does not use parallel ECM units, it achieves a higher throughput compared to all other devices. They report an improvement factor of 15.6 compared to the implementation of Gaj *et al.* We remark that the design does not compute the first point doubling $Q = 2P_0$ in hardware and needs the host to pre-compute this.

While the idea of the horizontal and vertical pipeline is very promising, the actual placement and routing on the chip using 100% of the DSP hardcores may generate problems. If these occur, they can - in most cases - be resolved by manual placement of the elements. As the design runs at a frequency of the modular multiplier (220 MHz), we assume that the internal high speed data paths were used. This internal connection is column-wise from bottom to the top DSP. The arrangement of the DSP elements on the FPGA suggests that this internal connection ends and a pipeline routes the (intermediate) result to the next column of DSPs. As the paper does not mention the place and route efforts in detail, we were unable to comprehend the measures used to avoid these problems.

Röpke presented 2008 another implementation to this challenge in [Röp08]. He tried to merge the ideas from the previous designs to create an arithmetic unit exploiting DSP hardcores with two multipliers and one addition and subtraction unit. It uses a 17×136 bit multiplier as a trade-off between an unrolled and a serial multiplier and works with the “Modular Multiplication with Quotient Pipelining” algorithm, presented by Orup [Oru95]. His architecture was created for the modified COPACOBANA cluster using Virtex-4 SX35 FPGAs with speed grade 10 and factored 151 bit integers. He presented a working design for the first phase and estimated the second phase’s results. His results showed an improvement to Gaj *et al.* but do not consider the fully pipelined implementation. After publication, the design was updated and the improvements compared to Gaj *et al.* became less significant. As with the implementation by Gaj *et al.*, we refer to these results.

We remark that the different designs cannot be compared as they use different FPGA devices and factor different bit sizes. To compensate these differences, the authors scale the results to bring them into perspective. Another difficulty is the cost-effectiveness ratio, the papers use different pricing models: buying a large amount of FPGAs, using EasyPath FPGAs⁸ or using the price of a single unit.

⁸see <http://www.xilinx.com/products/easypath/index.html> for more details.

For a consistent pricing model and fair comparison, we refer to the official Xilinx distributor Avnet Electronics Marketing⁹. The information are as of November 4th, 2009 for single purchase without discount and are rounded up to the next dollar. Due to the lack of information on the FPGA packaging for different designs, the cheapest commercial packaging was used.

Table 5.1 summarizes the different published implementations. In Chapter 7, we use this Table to compare our results without scaling.

Please note that not all papers mention the place and routing results and actual tests on an FPGA board. We were unable to resolve in all cases whether the frequencies are derived from the arithmetic unit, a single ECM unit or the complete design - and if they are post synthesis or post place and route timings.

Aside from the FPGA approach, other hardware-based ECM implementations were presented recently. These implementations exploit either graphic cards or Cell processors. Recently Bernstein *et al.* presented an efficient implementation of the elliptic curve method in [BCC⁺09], using a GTX 295 GPU clocked with 1242 MHz. Their design uses a higher smoothness bound $B_1 = 8192$, processing a larger scalar k of 11797 bits. Instead of Montgomery curves they used Edwards curves in their implementation. While they target only the first phase, the results of 4928 calculations per second is very promising. We will briefly compare our results with this implementation in Chapter 7.

⁹see <http://em.avnet.com/>

		2005	2006, revised 2008					2007	2008	
		XCV2000E-6	XCV2000E-6	XC2V6000-6	XC3S5000-5	XC3S1600E-5	XC4VLX200-11	XC4VSX25-10	XC4VSX35-10	XC4VSX35-10
Design	FPGA price (\$)	2,582	2,582	5,848	108	69	7,564	298	468	
	max Bitsize	198	198					135	151	
	Max. ECM Units	3	7	13	13	5	24	1	8	
	Max Frequency	38	47	120	80	96	104	220	174	100
	Slices (%)	18%	94%	90%	99%	94%	70%	58%	n/a	60%
	bRAM (%)	81%	94%	90%	99%	94%	70%	24%	n/a	48%
	DSPs (%)	-	-	-	-	-	-	100%	91%	
Operations (cycles)	Addition	78	41					1	24	
	Subtraction	52	41					1	24	
	Multiplication	666	216					1	66	
	Phase 1: $2P, P + Q$	7350	1212					n/a	377	392
	Phase 1: $P + Q$	4410	924					n/a	445	-
	Phase 1: $2P$	3608	n/a					n/a	287	-
	Phase 1: $2P, P + Q$	8018	1428					n/a	n/a	
Estimates	Phase 1	263ms	35ms	14ms	21ms	17ms	16ms	63μs	3ms	5ms
	Phase 2 (30)	462ms	81ms	32ms	49ms	41ms	37ms	-	-	
	Phase 2 (210)	-	40ms	16ms	24ms	20ms	18ms	-	3ms	-
Results	[Phase 1/s]	9	196	936	624	285	1,448	16,000	2,664	1,608
	[Phase 1/s per 100\$]	0	7	16	577	413	19	5,369	569	343
	[ECMs/s]	3	91	429	286	130	696	-	1,328	-
	[ECMs/s per 100\$]	0	3	7	264	188	9	-	283	-

Table 5.1.: Results of the related work from the latest revisions. Parameters are $B_1 = 960$, $B_2 = 57000$, and a scalar k of 1375 bits. The prices are as of November 4th, 2009 for single purchases without any kind of discounts.

6. Implementation

In this chapter we present our implementation of the elliptic curve method. It is based on the prototype by Röpke and also targets 151 bit integers. His proof of concept implementation mapped the structure from Pelzl *et al.* and Gaj *et al.* to a DSP based arithmetic unit. Nevertheless, it implemented only the first phase and had several performance issues. In this implementation, we worked out strategies to increase the number of parallel ECM computations, increase the clock frequency and present the complete design computing both phase 1 and phase 2. If not stated otherwise, we use the following ECM parameters of $B_1 = 960$, $B_2 = 57000$ and $D = 210$ to evaluate our design, e. g., clock cycles per operation or the number of occupied memory blocks.

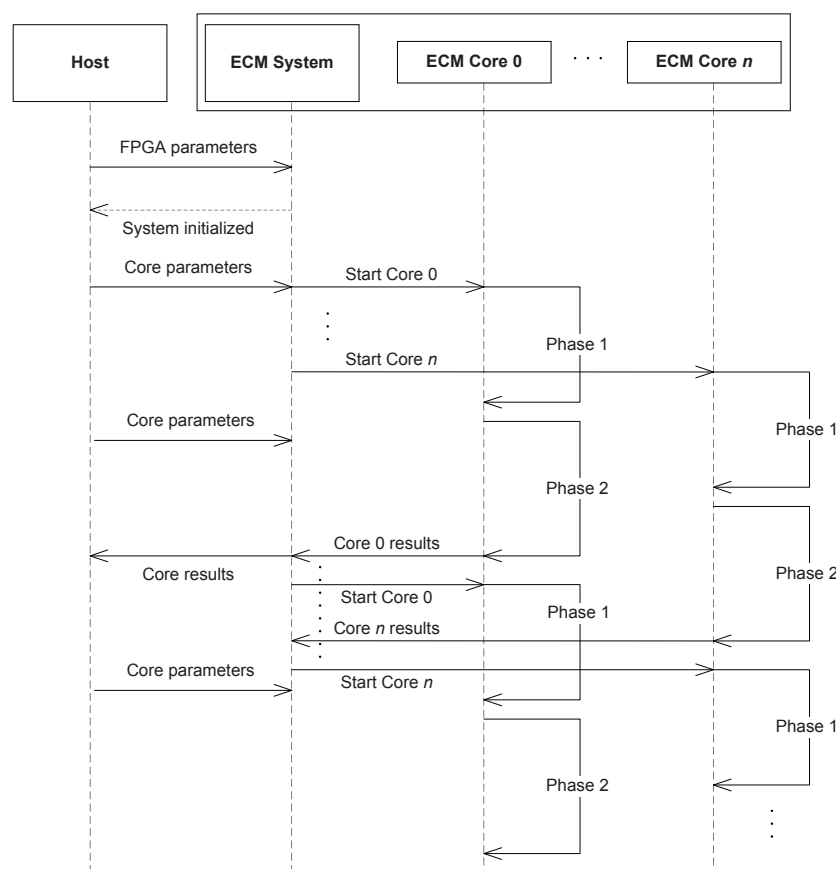
This chapter is divided into four sections. In the *System-Level View* we describe the ECM system, the necessary parameters and the interaction between the COPACOBANA and the host PC. The *Component-Level view* focuses on the FPGA and explains the architecture in detail. We give an overview of the clock management, bus design and the layout of the computation units. As the arithmetic-logic unit is the core of our design, we examine the components of the ALU in the *Low-Level View*, including detailed information on the memory management, the instruction ROM and the underlying modular arithmetic units. The chapter ends with the implementation of the *Elliptic Curve Method* using the ALU. In this section, the suggestions by Gaj *et al.* are analyzed and improvements - based on the evaluation of our design - are proposed.

6.1. System-Level View

We start with the description of the ECM design and the main hardware components: a host PC and the COPACOBANA.

The COPACOBANA cluster is equipped with 32 Virtex-4 FPGAs, each running a fixed number of *ECM cores*. All of these units use the same set of phase 1 and 2 parameters per FPGA. Nevertheless, we can configure each FPGA to use different smoothness bounds B_1 and B_2 to adjust the elliptic curve method, i. e., if all prime factors were found together and the result has been the composite number itself.

Figure 6.1 shows the interaction between the host and the FPGAs and - per FPGA - between the *ECM system* and the single ECM Cores: Each core is fed with data and



We need the host PC to generate the scalar k and the bit table *primetable* (see Section 6.4) depending on the ECM parameters per FPGA. Afterwards, we generate elliptic curves and calculate algorithm-specific moduli for the modular arithmetic units (see Section 6.3.3). These parameters are transferred to the COPACOBANA via TCP/IP protocol and the host waits for the results. When the computation is complete, these results are send back and the host evaluates them. As the host controls the data flow, the program ensures that each FPGA has enough data to

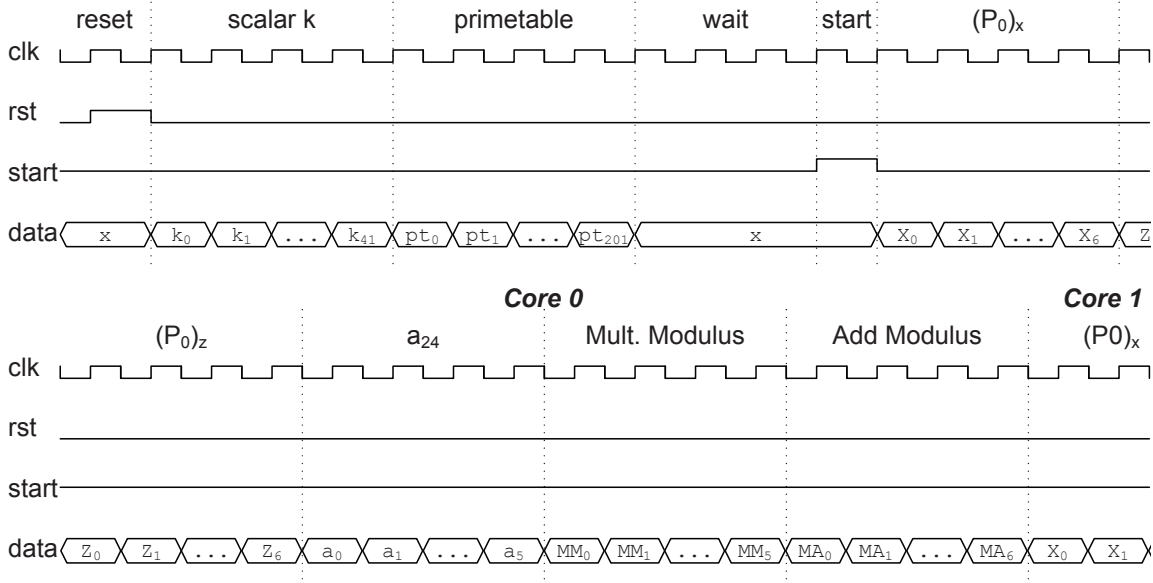


Figure 6.2.: Timing diagram of the ECM system (one FPGA)

keep the ECM system busy.

The COPACOBANA needs to be initialized once: We load each FPGA with the bitstream of the corresponding ECM system and instruction Sequence. Figure 6.2 shows the timing diagram of one FPGA's ECM system: After an initial reset, the FPGA needs the phase 1 parameter k of 42×32 bit blocks, followed by the phase 2 parameter *primetable* of 202×32 bit blocks to initialize the ECM system. When the parameters are stored, the system waits for a start signal. Once the signal is given, it requests the parameters for each core: six blocks for the x -coordinate and the z -coordinate of the starting point, five blocks for the modified curve parameter $a_{24} = \frac{a+2}{4}$ and the modified multiplication modulus $\tilde{M}$, followed by six blocks for the modular addition modulus $\tilde{M}$.

6.2. Component-Level view

In this section we describe the layout and design architecture of the ECM system and the interaction between the components.

To speed up the computation, one of the ideas we realized were different clock domains on the FPGA. Using the COPACOBANA framework, each FPGA is connected to the COPACOBANA Bus with 25 MHz (BUSCLK). We increase this frequency to

100 MHz and use it as the FPGA-wide system clock (SYSCLK). Then, we further increase it to 200 MHz and use this fast clock for our arithmetic operations.

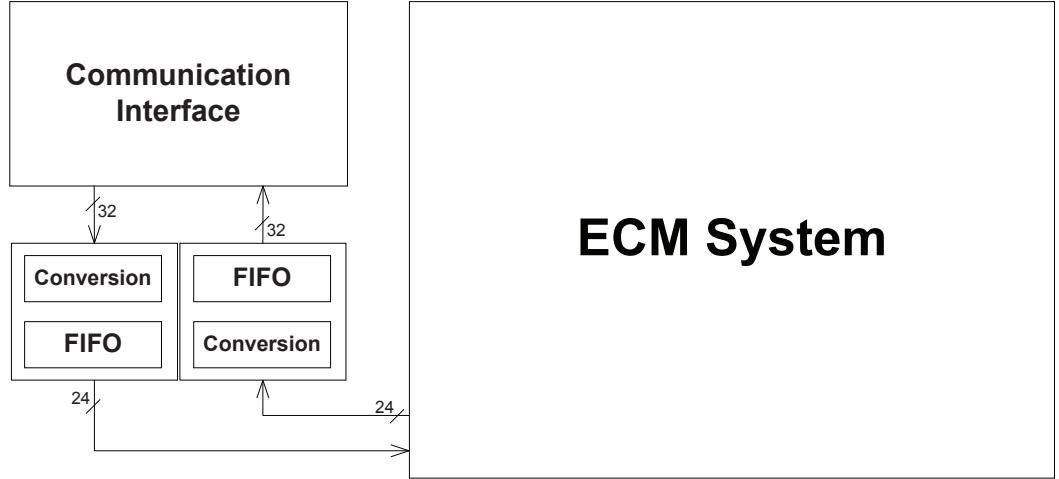


Figure 6.3.: FPGA layout showing the communication interface and the data bus between this interface and the ECM system

The layout of the FPGA and the external components are presented in Figure 6.3: The FPGA consists of the communication interface, two buffers and the ECM system itself.

The COPACOBANA uses a 32 bit Bus to transfer data to the FPGA, while the ECM system on the FPGA uses different data widths: to efficiently access the phase 2 parameter *primetable*, we need to store it row-wise and, as seen later in Chapter 6.4, need (at least) 24 bits. We also need to split all integers processed by the digital signal processing cores into 17 bit blocks. Thus, we use a global 24 bit system bus and - inside the arithmetic unit - a local 17 bit data path.

To minimize the number of required packages per ECM core, the host PC encodes the numbers in 32 bit blocks. The necessary conversion to 24 bits and 17 bits (and vice versa) is done by a conversion module. We use this conversion before incoming or outgoing data is stored in a FIFO buffer.

We remark that if the data transfer rate between the host and the FPGA is high enough, this conversion module can be removed: In this case, the host has to encode the data into the lower 24 bit.

Figure 6.4 shows the layout of the ECM system. It consists of a Data Request Scheduler, the system bus and a fixed number ECM cores. Depending on the size of the communication interface, the conversion module and different placements on the FPGA, up to 24 ECM cores can be placed.

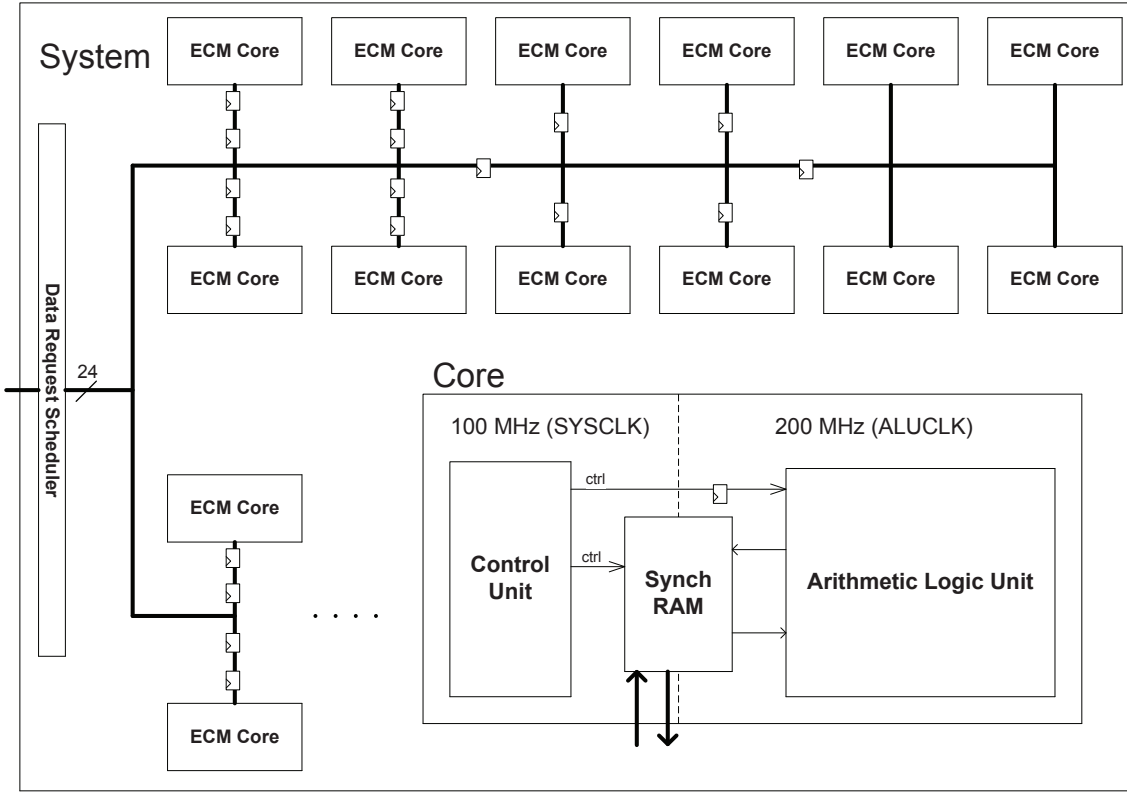


Figure 6.4.: Layout of the ECM system and ECM core

The system bus is implemented as a buffered tri-state bus and connects to the synchronization memory (SynchRAM) of each core. The number of buffering stages can be adjusted easily to optimize the system frequency in different design placements. To develop a flexible design, we use the Data Request Scheduler to manage the I/O communication and timing depending on the current bus design and buffers.

In the lower part of the Figure, we outline an ECM core in detail: It contains a Control Unit, the Arithmetic-Logic Unit (ALU) and the synchronization memory. Each core is split into two clock domains: The Control Unit uses SYSCLK to control the ALU, which uses the fast ALUCLK. In order to cross these domains, i.e for receiving the ECM parameters or sending the results, an asynchronous BlockRAM is used. As the ALU uses a shorter internal datapath, the synchronization memory connects only to the lower 17 bits of the system bus.

The control unit contains another BlockRAM which functions as global memory where the parameters k and *primetable* are stored for local access. The second part of this memory contains a sequence of ALU settings, which is used in several stages of the state machine which controls the ECM algorithm (see Chapter 6.4).

The last unit is the heart of the ECM core: the arithmetic-logic unit. It uses most of the dedicated FPGA hardware and implements a complex instruction sequence to control the basic modular arithmetic units and perform all ECM computations. We will explain this unit in the next section.

6.3. Low-Level View

In this section, we describe the low level components in detail and focus on the arithmetic-logic unit: we describe the underlying structure and connected components, as well as the instruction encoding to control the different operations.

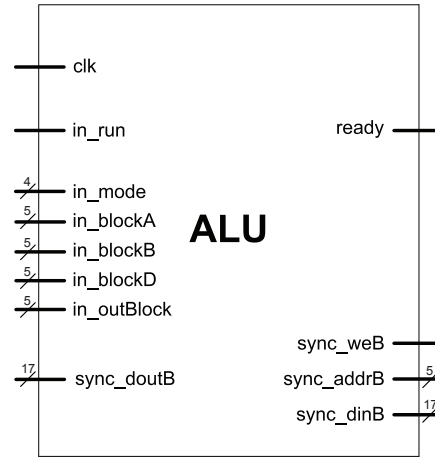


Figure 6.5.: Module description of Arithmetic-Logic Unit

We use the ALU to compute elliptic curve arithmetic, i.e., to compute a point addition on the curve. It provides the necessary functions to compute both ECM phase 1 and phase 2. Figure 6.5 shows the module with its input and output signals.

In this module, we operate using the ALUCLK for fast computations. In order to initialize the contained modules, the arithmetic unit is connected to the ECM core's synchronization memory via four sync ports. We use them to read (sync_doutB) and request or write to the memory using the output three ports sync_weB, sync_addrB and sync_dinB. We recall that synchronization memory truncates the signals from the system bus and feeds 17 bit data to the arithmetic unit. The reduced data path improves the local routing and increases the maximum frequency of the ALUCLK.

To start the unit, we select a mode from the list in Table 6.1 and use the in_mode port. The ALU operates on different memory blocks (see Section 6.3.1). We apply the correct configuration using in_blockA, in_blockB, in_blockD and in_outBlock

and set the `in_run` signal. The `ready` signal gives us information on the state of the computation.

	Mode	Description
Read Parameters	0	Read from synch, write $d = 1$
	1	Read from synch, write P_x
	2	Read from synch, write P_z
	3	Read from synch, write a_{24}
	4	Read from synch, write mul. modulus
EC Arith	5	Calculate $2A$, save result to <i>out</i>
	6	Calculate $A \oplus B$, use difference D , save result to <i>out</i>
	7	Calculate Montgomery Ladder: $out = A \oplus B$, $D = 2A$
Phase 2	8	Initialize next round
	9	Start the second phase
	10	Phase 2 step for two set bits
	11	Phase 2 step for one set bit
	12	Complete the second phase
	13	Write phase 1 and phase 2 results to synch

Table 6.1.: Modes of operation to select with `in_mode`

We use the first five modes to initialize the arithmetic unit for a new calculation. The data has to cross the clock domains and we need to control the synchronization memory accordingly. The opposite direction - i.e., in order to return the results - uses the last mode. The three arithmetic operations on elliptic curves we need in our computation are point doubling, point addition and the Montgomery Ladder. The arithmetic unit offers these functions as mode 5 to 7. As the second phase does not only use elliptic curve arithmetic, we need another set of operations. We will explain them in detail in Section 6.4.2.

Now that we have an overview of the operations the arithmetic unit can execute, we will describe its assembly: The ALU consists of one modular addition and subtraction unit, further denoted as MADDSUB, two modular multiplication units, further referred to as MMUL, and utilizes five BlockRAM units. Figure 6.6 shows these components: A dark shade indicates the arithmetic units, which will be explained in Chapter 6.3.3. The light shade indicates memory blocks.

We recall that the Virtex-4 SX35 contains eight columns of BlockRAM units surrounding the four columns of DSP hardcores. This distribution enables us to the memory as input/output buffer with both high frequency and short routing paths.

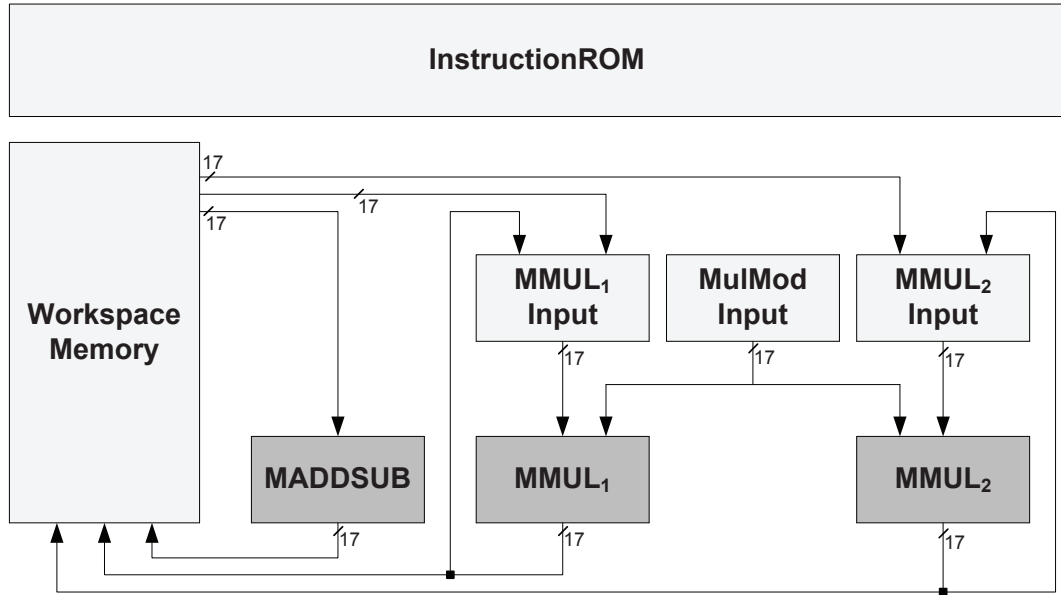


Figure 6.6.: Components of the arithmetic-logic unit

In our design, each modular multiplication unit uses a single BlockRAM as input buffer. We use another memory block to feed the modified multiplication modulus to these units asynchronously. This is convenient as parallel multiplications are not (always) computed synchronously. We remark that this BlockRAM unit can be changed into distributed memory easily. This allows us to implement the design on other FPGAs with a different memory segmentation. Furthermore, a different instruction sequence may be used to synchronize the multiplication units on cost of clock cycles. In this case, we can remove the (distributed) memory block completely.

In the next two sections we will present the two remaining BlockRAMs containing the workspace and the instructions. Afterwards we will introduce the basic arithmetic units.

6.3.1. Workspace Memory

We will now introduce the *Workspace Memory*: It is the central data storage the arithmetic unit operates on. As seen in Figure 6.6, the output signals of all arithmetic components are connected to this memory. Additionally, we can use data from the memory and write into the input buffer of both multiplication units. As only a small part is used for intermediate results, we will first describe the structure of the memory and then introduce the implemented memory management.

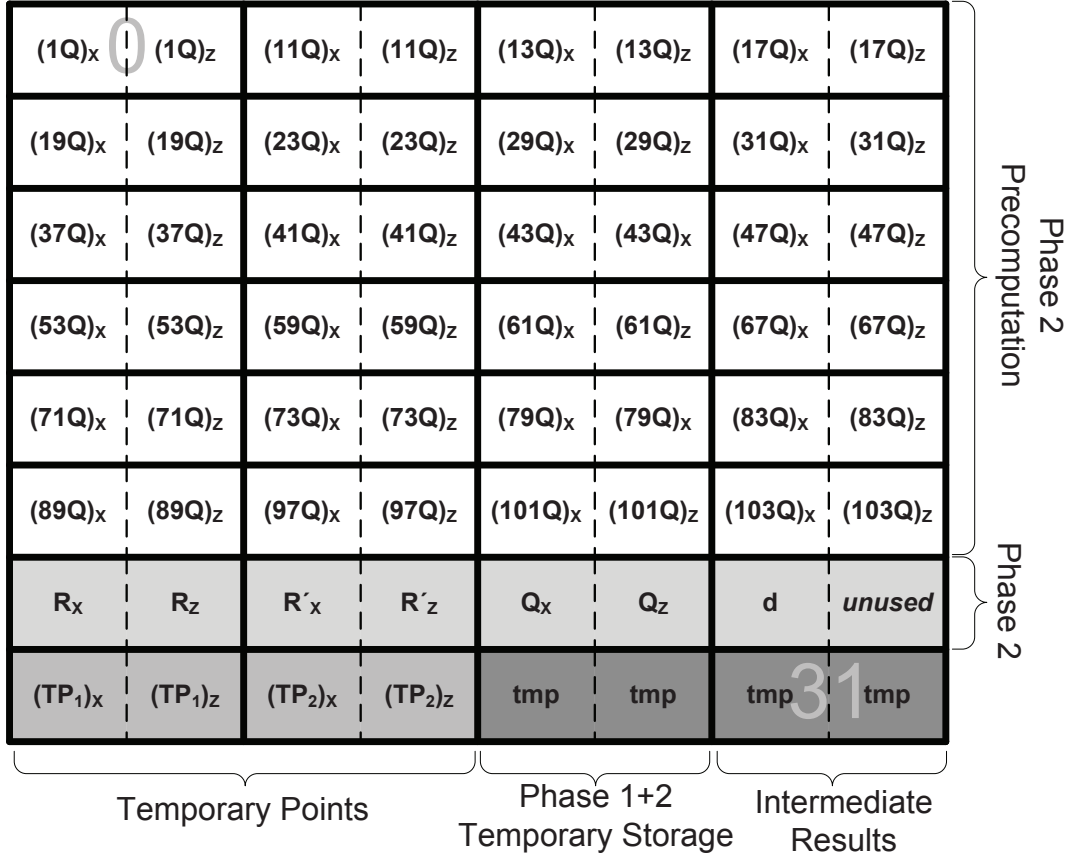


Figure 6.7.: Description of the *Workspace Memory* layout: it is partitioned in 32 storage blocks containing each two block cells.

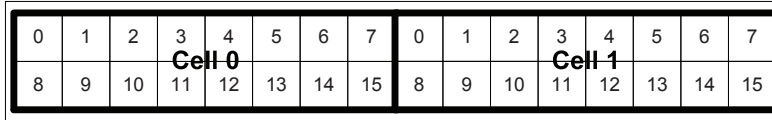


Figure 6.8.: The layout of one storage block: It contains 2 cells with each 16×17 bit blocks to save integers.

The *Workspace Memory* uses the complete 1024×18 bits of one BlockRAM unit to store data. It is divided into 32 storage blocks as shown in Figure 6.7. We define the top-left block as 0 and count row-wise. The layout of single storage block is shown in Figure 6.8: Each block B contains two cells, denoted as cell $(B,0)$ and $(B,1)$, which store up to 16×17 bit. This is realized as one $32 \times 2 \times 16 \times 18 = 1024 \times 18$ bit memory core and fits into one BlockRAM. Note that we use a similar layout in the input buffers and the synchronization memory, only using less storage blocks by

truncating the address bus.

In order to create a flexible memory management, we use address bus multiplexers. To decrease the size of these multiplexers, we address the 16 blocks using 4 bit counters. We achieve a better placement on the restriction that our arithmetic components now use a periodic input sequence and must not require special processing sequences.

We chose the design with the focus on efficient storage of elliptic curve points. If we write a point to storage block B , the first cell $(B, 0)$ will contain the x -coordinate, while the second cell $(B, 1)$ contains the z -coordinate. Figure 6.7 shows this memory partition and outlines the sections we write data throughout the ECM algorithm.

After the first phase is completed, storage block 0 contains the resulting point $Q_0 = kP_0$. We need the blocks 0 to 23 to store pre-computed points used in phase 2. The cells in storage block 24 to 27 are reserved for points and integers changed in phase 2, i. e., the result of the second phase. Note that the cell $(27, 1)$ is not used by the implemented instructions and available for modified instruction sequences. We use the last four storage blocks as temporary space: The pre-computation prior to phase 2 needs the space in block 28 and 29. Block 30 is reserved for intermediate values during the calculation of phase 1 and phase 2, while the last block is overwritten during elliptic curve arithmetic operations. We chose this memory partitioning to ensure data consistency in all ALU modes.

Using this flexible block management and the data consistency assertion, we can use the ALU in a sequence of instruction without explicitly controlling the internal processes and addressing the inner buffers. For example, given the memory content of Figure 6.7, we use the configuration $\text{in_mode} = 5$, $\text{in_blockA} = 11$, $\text{in_outBlock} = 28$ and compute the point doubling of $47Q$. When the computation finishes, the result $R = 2 \cdot 47Q = 94Q$ is available at block 28.

6.3.2. Instruction ROM

The logic inside the ALU is controlled by an instruction ROM. In the previous implementation, Röpke used 1128×147 bit instructions to control phase 1. This instruction memory was synthesized into 17 BlockRAM modules and used as a global Instruction ROM. The disadvantages were very long routing paths to all ECM units and resulted in a considerable reduction of the clock frequency. To counter this effect, he inserted several 147 bit registers as pipeline stages.

This design made it very difficult for us to extend the algorithm and include the second phase. In the following, we present a different approach to implement the

Port A:			
Synchronization	MMod	MADDSUB	MMUL ₁
53..51	50	49..46	27..18
Workspace			
45..28			
Port B:			
MMUL ₂		Control	
17..8		7..0	

Figure 6.9.: New instruction ROM design: Instructions are divided into a 36 bit and an 18 bit part, using a dual port ROM to read the instructions. Note that the Workspace and MMUL₁ Instructions are swapped in this figure for visualization purposes.

instruction ROM. This approach makes a local instruction ROM possible for each ECM core. Furthermore, we can remove the buffers and decrease the area consumption:

We allocate 341×54 bits and store the instructions of all 14 modes. To do this, we have to exploit the dual port architecture. We recall that dedicated memory blocks of Virtex 4 FPGAs support a data width of 1, 2, 4, 9, 18 or 36 bit for each port (see Chapter 2.2.1). As we need more bits per instruction, we split them into a 36 bit and an 18 bit part. This is outlined in Figure 6.9. Both parts are read using the dual ports in parallel. Using this alignment allows us to store all instructions in one 512×36 bit BlockRAM, while the normal approach will synthesize as a cascade of two BlockRAM units and implement a 512×54 bit memory.

The instructions are divided into multiple parts to control the synchronization RAM, MADDSUB, the Workspace Memory, both multiplication units and the instruction controller. Table 6.2 lists the instructions in detail. Most instructions are used to reset counters and units and to control the storage blocks of the different BlockRAMs. We included a block setup signal (`w_blockCtrl`) in the workspace control signals. Using this instruction, we can choose which block-information is passed to the workspace memory.

As listed in Table 6.3, there are seven different modes. In the (normal) mode 0, the workspace is addressed by (`w_blockA`, `w_cellA`) and (`w_blockB`, `w_cellB`). We can combine the ALU inputs using different control modes. After an analysis of the algorithms, we were able to realign the instructions and reduce the number of necessary combinations down to 7 without losing functionality or increasing the number of clock cycles.

In addition to the control signals for the main components, the instructions reserve the last block as information on the sequence itself. The most obvious signal is the `finish` flag which terminates an instruction sequence. As we analyzed the instruction sequence of a prototype implementation, we noticed that several instruc-

	Bit(s)	Name	Description	
Sync	53	s_cnt_rst	Reset address counter	
	52	sync_weA	Write to synchronization RAM	
	51	sync_block	Storage block (0 . . . 1)	
	50	mulmod_we	Write to MulMod Input Buffer	
MADDSUB	49	a_rst	Reset MADDSUB	
	48	a_cnt_rst	Reset address counter	
	47	a_in_complete	No more MADDSUB input	
	46	a_mode	0 - Addition, 1 - Subtraction	
Workspace Memory	45	w_ctrlA	dinA = MADDSUB (0) or MMUL1 (1)	
	44-42	w_blockCtrl	Configure storage block input	
	41	w_weA	Write to port A	
	40-36	w_blockA	Storage block for port A	
	35	w_cellA	Block cell for port A	
	34	w_weB	Write to port B	
	33-29	w_blockB	Storage block for port B	
	28	w_cellB	Block cell for port B	
MMUL 1 and 2	27	17	m_rst	Reset MMUL
	26	16	m_sqr	Calculate square (resolve W/R collision)
	25	15	m_cnt_rst	Reset address counter
	24	14	m_ctrlA	dinA = MADDSUB (0) or Workspace port A (1)
	23	13	m_weA	Write to port A
	22-21	12-11	m_blockA	Storage block for port A
	20	10	m_weB	Write to port B
	19-18	9-8	m_blockB	Storage block for port B
Control	7	fin	Current instruction sequence completed	
	6-1	duration	Cycles to keep current instruction valid	
	0	zerostate	Pre-Fetching: Is duration = 1 ?	

Table 6.2.: List of the different instruction signals and the instruction encoding.

w_blockCtrl	port A	port B
0	w_blockA	w_blockB
1	in_blockA	in_blockA
2	in_blockB	in_blockB
3	in_blockD	in_blockD
4	in_outblock	in_outblock
5	in_blockB	w_blockB
6	in_blockD	w_blockB
7	unused	unused

Table 6.3.: Storage block input configurations of the Working Memory using the w_blockCtrl control signal.

tions were constant for up to 106 clock cycles. To reduce the number of instructions stored in the memory, we removed these steady instructions using the amount of clock cycles as additional information. When our instruction is valid for exactly one clock cycle, the implemented pipeline and instruction pre-fetching mechanism requires the zerostate flag to ensure correct work.

We end this section and provide Table 6.4 as a summary of the different modes and the relation between the number of the instructions and the clock cycles: Out of the available 341 instructions, we use only 326 to implement the complete elliptic curve method architecture. These instructions cover a total of 2,665 clock cycles.

6.3.3. Modular Arithmetic Units

In this section we develop the basic modular arithmetic units which are based on the implementation by Röpke. First, we will examine the multiplication unit, as it is the more important and difficult unit, and end with the addition and subtraction unit. To develop our implementation, we start by reviewing the previous implementation and outline possible weaknesses in combination with the new ALU design.

Modular Multiplication Unit

Röpke introduced an implementation of the montgomery multiplication using DSP hardcores. He chose to implement a variant of the Montgomery Multiplication, the *Modular Multiplication with Quotient Pipelining* by Orup. This algorithm was iteratively constructed by rewriting the original multiplication and presented as Algorithm 4 in [Oru95]: Orup proposed different variants to simplify the quotient han-

	Mode	Description	Instructions	Cycles
Read Parameters	0	Initialize $d = 1$	4	13
	1	Initialize P_x	4	13
	2	Initialize P_z	4	13
	3	Initialize a_{24}	4	12
	4	Initialize mul. modulus	4	12
EC Arith	5	Calculate $2P$	43	434
	6	Calculate $P \oplus Q$	60	501
	7	Calculate Montgomery Ladder Step	105	688
Phase 2	8	Initialize next round	9	24
	9	Start the second phase	9	132
	10	Phase 2 step for two set bits	37	392
	11	Phase 2 step for one set bit	23	262
	12	Complete the second phase	12	145
	13	Output results	8	24
		Total	326	2,665

Table 6.4.: Number of instructions per ALU mode and the cycles needed to execute the complete instruction sequence.

dling. This is of particular interest to hardware implementations because it allows continuous operations using DSP slices. The proposed algorithms require a word-wise multiplication, addition and shifts. The input is split into n blocks of k bits each. As seen in chapter 2.2.2, the Virtex-4 DSP Cores support these basic operations and provide an optimal platform to perform sequential computations without using fabric resources.

The particular algorithm used in the previous work and this thesis is a simplified version of Orup's "Modular Multiplication with Quotient Pipelining". It needs $n + 2 + d$ steps for a fixed delay parameter d . We set the pipelining parameter $d = 0$ and simplify the algorithm as shown in Algorithm 7.

The definition of $\check{M}$ shows the connection between the number of blocks n and the size of the modulus M : $M < 2^{k \cdot (n-1) - 2}$. To use the dedicated DSP cores, Röpke set $k = 17$. As the previous implementation, we choose $n = 10$ and reserve 170 bit for each intermediate result over a 151 bit modulus. This is necessary because the algorithm does not compute the last reduction step to save time.

Röpke used a parallel architecture for the multiplication with one DSP hardcore is used for each block. After an initial pipeline delay of six clock cycles, the first block

Algorithm 7 Modular Multiplication with Quotient Pipelining and $d = 0$

Input: Modulus $M > 2$ with $\gcd(M, 2) = 1$, $k, n \in \mathbb{N} : 4\tilde{M} < 2^{kn}$. $\check{M} := (\tilde{M} + 1) \operatorname{div} 2^k$, $\tilde{M} := (M' \bmod 2^k)M$, with $(-MM') \bmod 2^k = 1$. Integers A, B in Montgomery Form with $2^{kn}R^{-1} \equiv 1 \bmod M$ and $0 \leq A \leq 2\tilde{M}$, $0 \leq B \leq 2\tilde{M}$. b_i refers to the i -th k -bit block of B .

Output: $S_{n+2} \equiv ABR^{-1} \bmod M$ and $0 \leq S_{n+2} < 2\check{M}$

$S_0 := 0$

for $i := 0$ to n **do**

$q_i := S_i \bmod 2^k$

$S_{i+1} := S_i \operatorname{div} 2^k + q_i \cdot \check{M} + b_i A;$

end for

$S_{n+2} := 2^k S_{n+1}$

of the result is ready after $n \cdot \frac{n}{2}$ clock cycles. In total, $6 + n \cdot \frac{n}{2} + n$ cycles are necessary to obtain all blocks of the result.

This implementation posed two difficulties in combination with our new ALU design: we need a cyclic input sequences, if we want to use counters in order to reduce the address bus width. But the timing diagram of the multiplier¹⁰ shows an input sequence which concatenates not consecutive blocks. This means we cannot collect the blocks in a shift register, because concatenated inputs $[a_0:a_5]$, $[a_1:a_6]$, $\dots$ are required. The other problem is the number of DSP cores used. Because the ALU uses two modular multiplication units, this approach already reserves more than 10% of the available DSP units.

In this implementation we try to counter this limitation and use a fixed amount of three DSP cores to sequentially calculate the result. Using a sequential approach also enables us to change the input dependencies and remove the special block arrangement.

Figure 6.10 shows the new multiplication module. The multiplier uses the inputs A , B and M , each 17 bits wide, and is clocked by the $ALUCLK$. After the computation is complete, the result is available on the output port $result$.

Figure 6.11 shows the configuration of the three DSP cores. Note that the feedback path which connects the result block to the q_i input is not included. The index i denotes round values while the index j represents the current 17 bit block of the number during the sequential computation.

The first DSP calculates $q_i \cdot M_j + S_{ij}$ and passes the result to the second DSP using the fast internal data path. This core adds $A_j \cdot b_i$ and the result leaves the internal

¹⁰see Figure 7.8 in [Röp08]

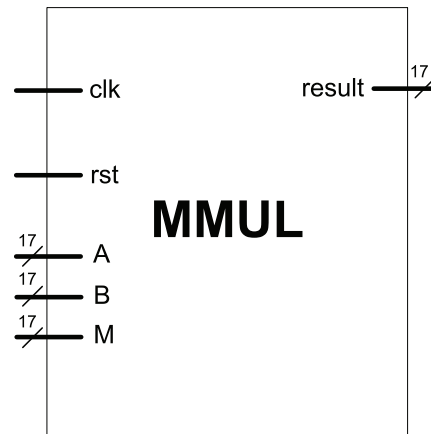


Figure 6.10.: Input/Output ports of the new module MMUL.

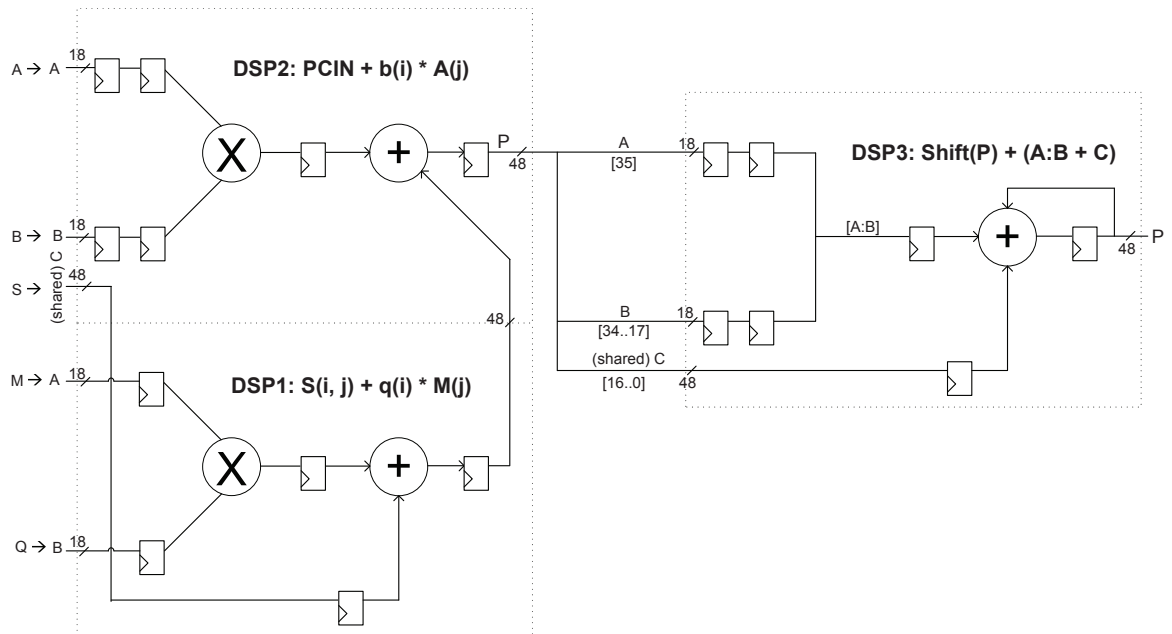


Figure 6.11.: DSP configuration of the modular multiplication unit

data path. It is split into three parts and fed into the third DSP, which handles the necessary carry operation. Using this configuration computes the Algorithm 7.

The timing of our new multiplication unit is described in Figure 6.12. After a reset clock cycle, the first input iteration starts and needs $n + 1$ clock cycles to complete. Note that one additional clock cycle per iteration was added and the data of block $n + 1$ is assumed to equal zero. As we have already seen, the ALU design can store up to 16 blocks per number without hardware changes and thus meets this

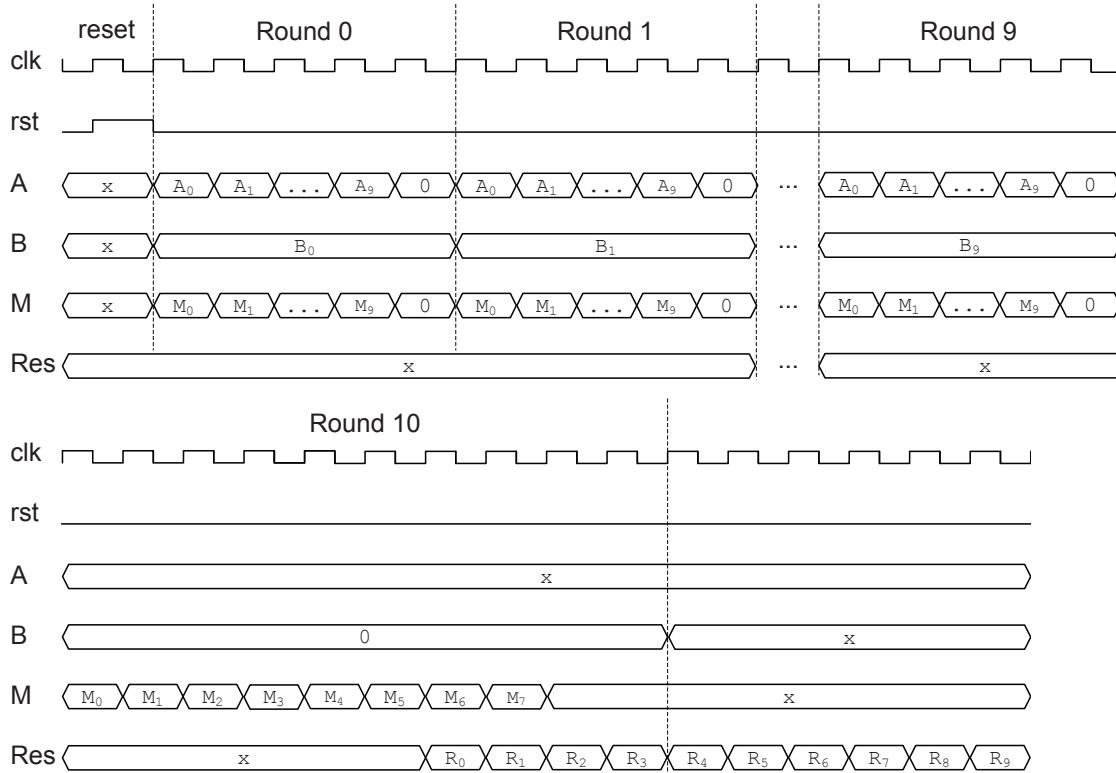


Figure 6.12.: Timing diagram of the modular multiplication unit

requirement. After n iterations there is no more input data. Because the 11th block $b_{11} = 0$, the input A cannot modify the result, which passes through 6 pipeline registers - three at DSP1 and 2 plus additional three at DSP3 - and is valid for n cycles. Thus our multiplier needs a total of $n \cdot (n + 2) + 6$ clock cycles.

To summarize our achievements, we changed the multiplication from parallel to sequential and reduced the number of DSPs from n to a fixed amount of three DSPs on the cost of clock cycles. This time-area trade-off is shown in Figure 6.13. The fixed amount of DSPs gives us another advantage: the design is independent of the input bit length. Our new architecture is valid for any number of 17 bit blocks $5 \leq n$. This lower bound comes from the calculation of q_{i+1} . If we use less blocks in the multiplication, we need to implement delays to adjust the timing. To use more than 5 blocks, we can implement the storage of q_{i+1} as a shift register. The size of this register is the only part in the design we need to adjust when we vary the number of blocks. Even though there is no upper bound in the architecture, the memory management design limits us to $5 \leq n \leq 15$.

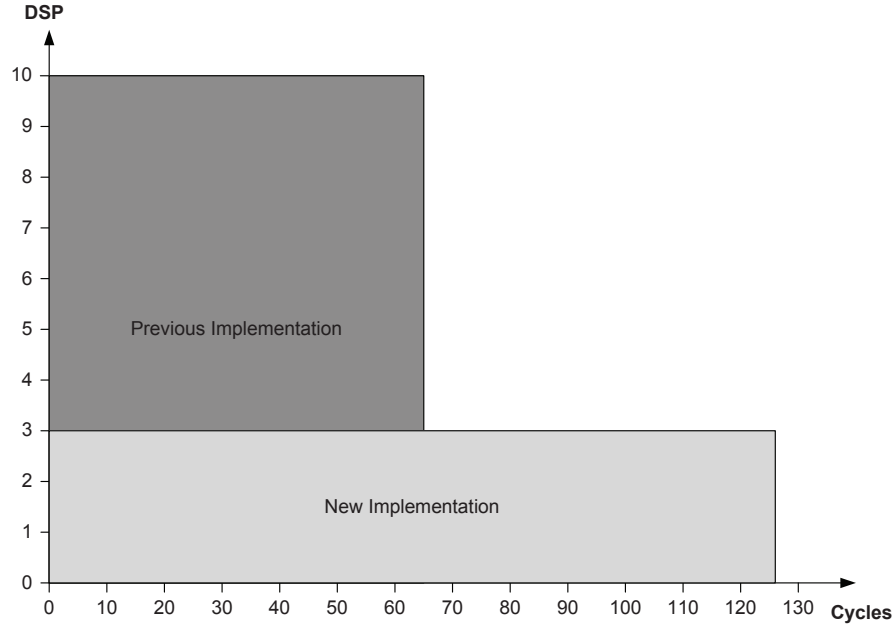


Figure 6.13.: Parallel vs. sequential: The implementation by Röpke used 10 DSPs and needed 66 clock cycles for one multiplication, while the new unit needs only 3 DSPs on cost of 60 more cycles to compute the result.

Modular Addition/Subtraction Unit

The second modular arithmetic unit calculates the modular addition and subtraction. As we want to feed results from a multiplication directly to this unit, we have to process 170 bit numbers and reduce them by $2\check{M}$. Before we start with our design, we will review the module by Röpke:

The module MADDSUB uses two DSP cores. As the DSP cores support a concatenation of the (signed) 18 bit inputs A and B and add the (signed) 48 bit input C as a native operation, they work with $l = \frac{n}{2} = 5$ 34 bit input blocks. After processing the block j , the first DSP passes the intermediate result $R_j = A_j \pm B_j (+ \text{CIN})$ to the second DSP for the reduction $S_j = R_j \mp M_j (+ \text{CIN})$. Both operations work in parallel, shifted by one clock cycle. At the end of the computation, the unit evaluates the borrow bit of the subtraction (either by the main or reduction DSP) and determines the correct output.

For carry handling, each input block was processed twice. As a result, the input and output were both valid for two clock cycles¹¹. This lead to n clock cycles to calculate the complete result, one cycle as a delay for the reduction plus $2n$ additional

¹¹see the Timing diagram, Figure 7.11 in [Röp08]

clock cycles to output the result - $2n + 3$ in total.

The addition is not the critical operation in terms of clock cycles. Gaj *et al.* proposed formulae to calculate the ECM, requiring that two additions must not exceed the time of one multiplication and the implementation by Röpke fulfills this bound. Still, there are two reasons for a redesign: After publication, the module was adjusted and contained a higher maximum delay, as well as a memory component as the output buffer. Additionally, after all input blocks were processed, the unit required all inputs to save the last value until the result was read. This did not work with the consecutive and cyclic input required by our design.

Now we will present the new implementation of the modular addition and subtraction unit using DSPs. After reducing the ALU bus to 17 bit, we need to position the MADDSUB module in a wrapper. It handles the conversion from 17 bit to 34-bit blocks and feeds the inner module. Figure 6.14 shows the ports of both the outer and inner unit.

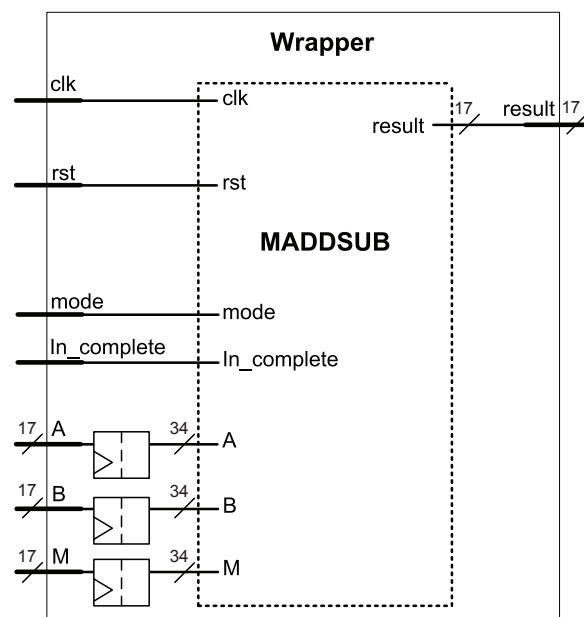


Figure 6.14.: Input/Output ports of the new module MADDSUB.

The input `mode` switches between addition (`mode = 0`) and subtraction (`mode = 1`). The integer inputs `A`, `B` and `M` are passed to the module directly from the memory. To solve the problem with the previous design, a comparison with a counter can “freeze” the input. Instead of this comparator, we spent one bit in the instruction ROM to signal the end of our input and connect this to the `in_complete` port. The inner module uses a shift register as output buffer and passes the 17 bit results to

the result port. Note that this output storage shift register is the only part in the fabric which will change if we vary the number of blocks n in the implementation. The changed behavior of `in_complete` is realized by a new, adjusted instruction sequence.

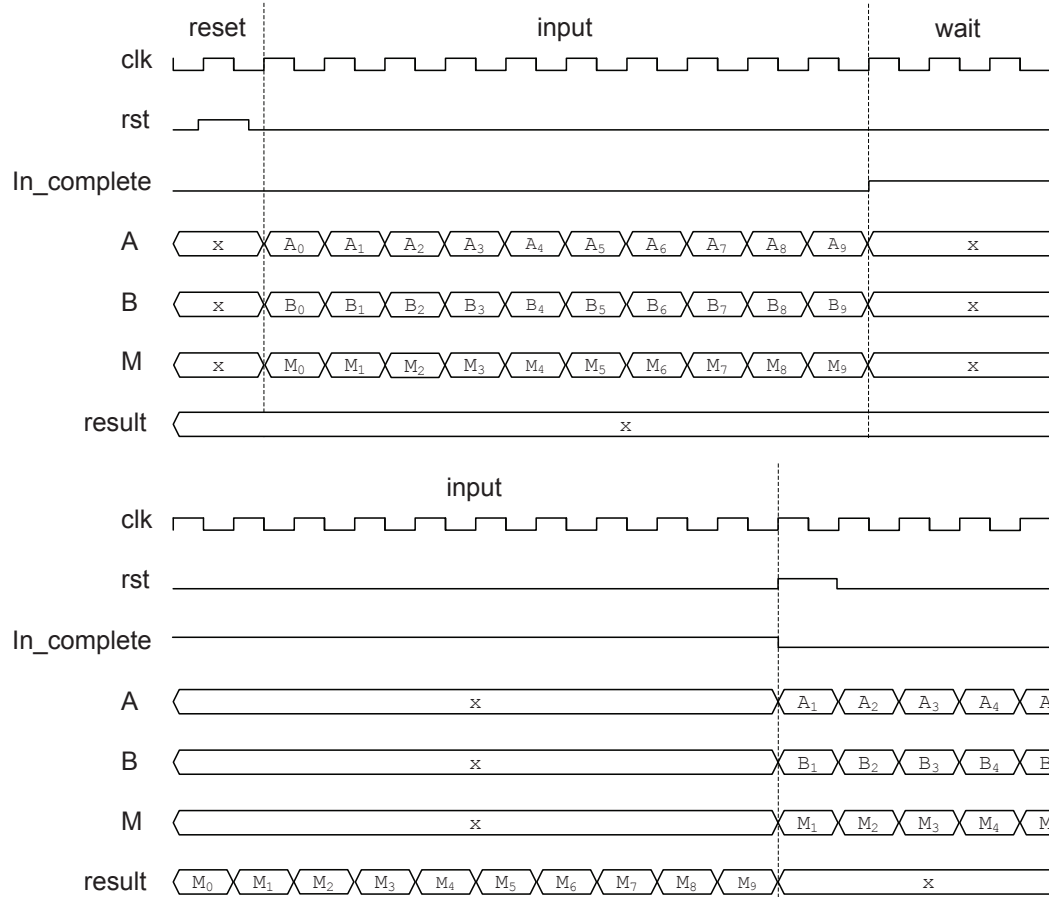


Figure 6.15.: Timing diagram of the modular addition and subtraction unit

Figure 6.15 shows the timing diagram of the wrapper module¹². After the ALU passes all input blocks to the wrapper, the instruction ROM sets the `in_complete` flag. Afterwards, the wrapper does not care about the inputs and returns the correct result after a three clock cycle delay. This leads to a total number of clock cycles of $2n + 3$ and equals the amount of time the unit by Röpke needed.

Using the wrapper gives us the possibility to exchange the inner addition and subtraction module without changing the behavior of the surrounding system. While

¹²In our implementation, the input M is connected to the result signal by a multiplexer, using the `in_complete` signal as control. As this detail is only important for local optimizations on the initialization process, we did not include it in the figure

this is normally not necessary, it is crucial in our case: our adapted and optimized implementation removed the BlockRAM buffer and changed the logical functions. The result was a module running at the FPGA's maximum frequency with a small area consumption. A problem was revealed when the complete system was ready and 24 ECM cores had to be placed in the FPGA: the module uses the shared C input. We recall that each DSP tile of the Virtex 4 contains two DSP slices. They share one input port C. As we need two multiplication units per ALU and one addition and subtraction unit, we use a total of eight DSPs. We designed the modular multiplication unit to use three DSPs with two ports C (see Figure 6.11), which makes four shared C ports per ALU. If the MADDSUB module tries to use another shared port, our ALU will need five DSP tiles and blocks ten DSPs. Note that only Virtex 4 FPGAs contain a shared input per DSP tile and thus we can use this implementation with the latest devices such as the Virtex-5, Virtex-6 or Spartan-6 families.

To solve this problem, we implemented a second MADDSUB unit. It is more complex and uses two different OPMODE configurations per DSP. The main idea is to use the concatenated input $[A:B]$ to load one integer per clock cycle and use this value in the next cycle to perform an operation using the internal feedback path. We can configure the DSPs to use the following modes: $A:B + CIN$ and $P \pm (A:B + CIN)$. Even though this approach needs more space and reduces the max frequency slightly, it matches our requirements and is suitable for the modular addition and subtraction in the proposed arithmetic unit.

6.4. Elliptic Curve Method

In this section we will present the implementation of the elliptic curve method using the operations of the arithmetic unit. We start with the first phase, but focus on the second phase, as it is the more complex part. Gay *et al.* suggested an effective way to implement both phases of the ECM using two modular multiplication units and one modular addition and subtraction unit. Our implementation is based on these suggestions, but tries to improve and optimize them using our design.

We show the main steps of the control state machine in Figure 6.16. The different shadings indicate different sources for the instruction. The FPGA initialization sequence is hard-coded into the bitstream. The sequence mentioned before is processed at the white states, which read from the global memory. This gives us the possibility to change the pre-computation sequences without modifying design itself. In order to compute phase 1 and phase 2 of the ECM, we execute a series of steps and parse the ECM parameters k and *primetable*. This is done by the second

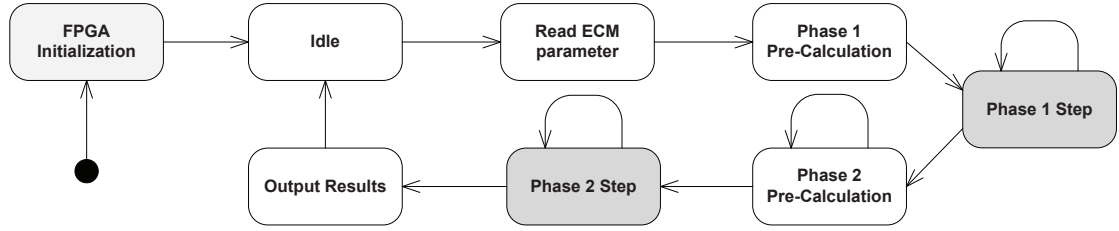


Figure 6.16.: State Machine of the control unit

state machine parallel to the execution of the current ALU calculation to ensure minimal delay. The following sections contain more details about the pre-computation sequences and the state machine computing for both phases.

6.4.1. Phase 1

This implementation uses the Montgomery Ladder (see Chapter 4.3, Algorithm 5) to compute the first phase the elliptic curve method. We use the proposal published by Montgomery and parse the scalar k bitwise and execute one step of the ladder each time.

The ALU mode 7 computes one step of the Montgomery Ladder. The ALU implementation for phase 1 is shown in 6.17. The Montgomery Ladder algorithm ensures in each step $P - Q = P_0$. The first phase allows us to choose the point P_0 arbitrary and by using $(P_0)_z = 1$ we can save one multiplication in the concurrent point addition and point doubling formula. Note that this is only possible in the first phase.

We recall Figure 6.16: after the ECM parameters are parsed, the phase 1 pre-calculation starts. Using the Montgomery Ladder, this is a simple point doubling to calculate $2P_0$ from the starting point. Afterwards, the main calculation begins.

Figure 6.18 shows our state machine to process phase 1 in the current implementation. During the pre-calculation we search for the first bit of k in the memory. Later during the first phase, we use a shift-register to access the next bit of the current memory block of k . If necessary, we can always request a new block from the ECM core's memory. As soon as the arithmetic unit changes ready to busy, we configure the upcoming calculation in the state `phase1_set_alu_signals` using the current bit of the scalar k (`parameter_bit`). This bit determines the block addresses and ensure that Q is always stored at block 0, while P is always written to block 1. The first phase completes after $\log_2 k - 1$ Montgomery Ladder steps.

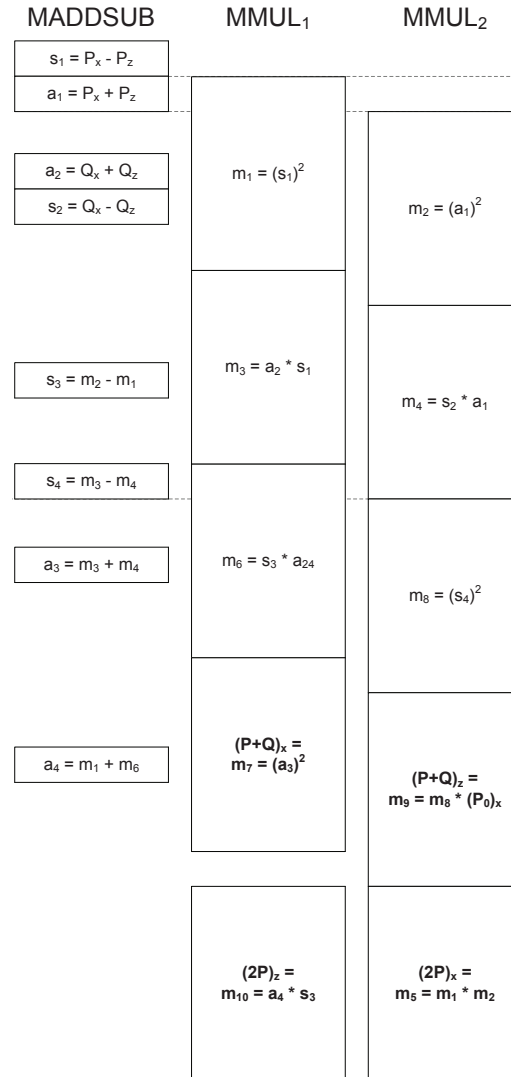


Figure 6.17.: Implementation of the Montgomery Step using P_0 as starting point.

6.4.2. Phase 2

To implement the second phase pre-calculations are required to reduce the number of iterations. Gaj *et al.* suggest the following Algorithm 8 for the second phase. Pelzl *et al.* suggested that “ D should consist of small primes in order to keep $\varphi(D)$ ”¹³ and further suggest $D = 6$, $D = 30$, $D = 60$ and $D = 210$. As our FPGA contains a large memory, we choose $D = 210$. From now on we use this value in our analysis.

We will start with a review of the algorithm and then outline the parts we changed

¹³see [PSK⁺05], p. 4

Algorithm 8 Standard Continuation Algorithm of phase 2 from [GKB⁺06]

Input: Composite integer N to be factored, elliptic curve $\mathcal{E}$, $Q_0 = kP_0$ as result of phase 1, B_1 smoothness bound for phase 1, B_2 smoothness bound for phase 2, $B_2 > B_1$, parameter D determines the trade-off between computation time and the amount of memory required, D is assumed to be even.

Output: factor q of N , $1 < q \leq N$ or FAIL

Precomputations:

```

1:  $M_{MIN} \leftarrow \lfloor (B_1 + \frac{D}{2})/D \rfloor$ ,  $M_{MAX} \leftarrow \lceil (B_2 - \frac{D}{2})/D \rceil$ 
2: clear GCD_table, clear  $J_S$ 
3: for each  $j = 1$  to  $\frac{D}{2}$  step 2 do
4:   if  $\gcd(j, D) = 1$  then
5:     GCD_table[j] = 1
6:     add  $j$  to  $J_S$ 
7:   end if
8: end for
9: clear primetable
10: for each  $m = M_{MIN}$  to  $M_{MAX}$  do
11:   for each  $j = 1$  to  $\frac{D}{2}$  step 2 do
12:     if  $mD + j$  or  $mD - j$  is prime then
13:       prime_table[m, j] = 1
14:     end if
15:   end for
16: end for
17:  $Q \leftarrow Q_0$ 
18: for  $j = 1$  to  $\frac{D}{2}$  step 2 do
19:   if GCD_table[j] = 1 then
20:     store  $Q$  in  $S$   $\{Q = jQ_0 = ((jQ_0)_x : (jQ_0)_z)\}$ 
21:   end if
22:    $Q \leftarrow Q + 2Q_0$ 
23: end for

```

Main computations:

```

24:  $d \leftarrow 1$ ,  $Q \leftarrow DQ_0$ ,  $R \leftarrow M_{MIN}Q$ 
25: for each  $m = M_{MIN}$  to  $M_{MAX}$  do
26:   for each  $j \in J_S$  do
27:     if primetable[m, j] = 1 then
28:       retrieve  $jQ_0$  from table  $S$ 
29:        $d \leftarrow d \cdot (R_x(jQ_0)_z - (jQ_0)_x R_z) \{R = (R_x : R_z)\}$ 
30:     end if
31:   end for
32:    $R \leftarrow R + Q$ 
33: end for
34:  $q \leftarrow \gcd(d, N)$ 
35: if  $q > 1$  then
36:   return  $q$ 
37: else
38:   return FAIL
39: end if

```

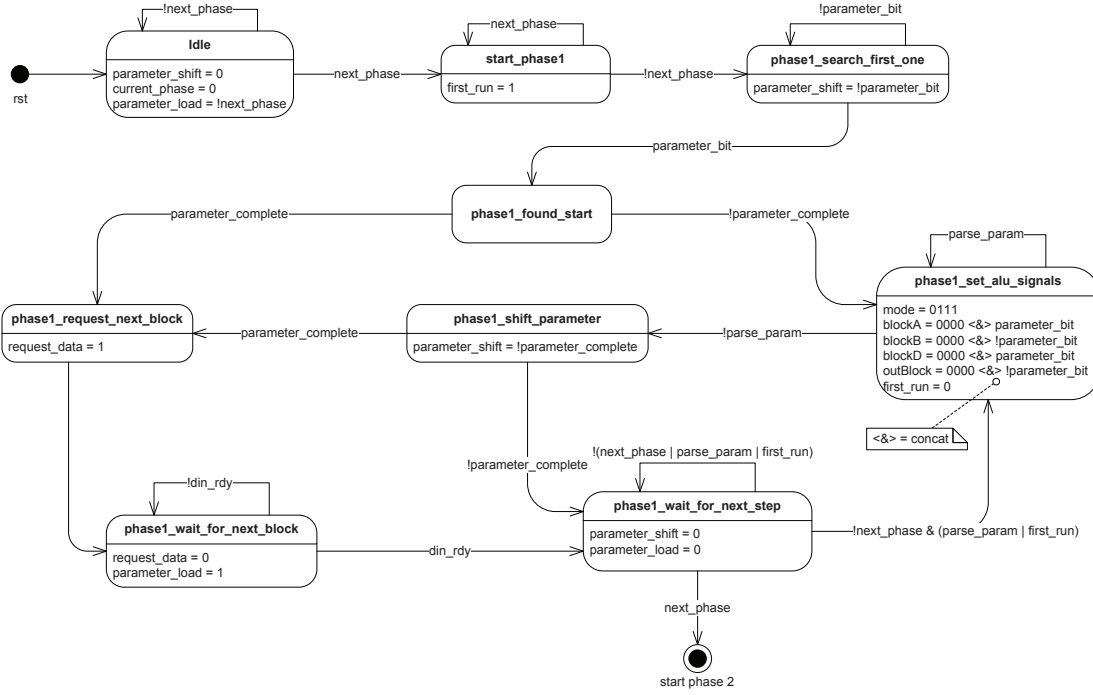


Figure 6.18.: Implementation of the Montgomery Step using P_0 as starting point.

to optimize the second phase. We recall that we want to express all primes between B_1 and B_2 as $\pi = mD \pm j$ for the integers $M_{MIN} \leq m \leq M_{MAX}$ and $1 \leq j \leq \lfloor \frac{D}{2} \rfloor$ co-prime to D .

Pre-computation

Gaj *et al.* suggest to pre-compute the values of j as they form a small set, as well as two bit tables for fast tests in hardware. These information can be re-used for multiple executions of the second phase, as long as the ECM parameters B_1, B_2 and D do not change.

As we use $D = 210$ in our design, we can calculate the set $J_S(D)$ from Algorithm 8 and obtain the following 24 numbers:

$$J_S(210) \leftarrow \begin{matrix} 1 & 11 & 13 & 17 & 19 & 23 \\ 29 & 31 & 37 & 41 & 43 & 47 \\ 53 & 59 & 61 & 67 & 71 & 73 \\ 79 & 83 & 89 & 97 & 101 & 103 \end{matrix}$$

These numbers conclude the multiples of the phase 2 starting point Q_0 we need to pre-compute and store in the memory. Note that this information is encoded in the $\lfloor \frac{D}{4} \rfloor = 52 \times 1$ (D is even) bit table `GCD_table`.

It is not easy, to include a prime test in hardware and check if $mD \pm j \in \mathbb{P}$. The proposed *primetable* solves this problem, as it allows us to determine if a prime is expressed by m and j . The algorithm generates this table in line 9 to 16: If $mD \pm j$ is prime, it stores a 1, otherwise a 0. This generates a $(M_{MAX} - M_{MIN}) \times 24$ bit table. We remark that the algorithm iterates over $1 \leq j \leq \frac{D}{2}$ instead of the elements of $J_S(D)$ and generates a larger table than necessary.

The last part in the pre-computation is the computation of the multiples jQ_0 . The proposed algorithm (lines 17-22) is a generic method and needs 52 point additions. Additionally, the main computation requires the following multiples $DQ_0 = 210Q_0$ and $M_{MIN}DQ_0 = 210M_{MIN}Q_0$ (line 25). Gaj *et al.* propose to use a similar algorithm as used in phase 1 with one additional multiplication (because $(P - Q)_z = (Q_0)_z \neq 1$)¹⁴.

Instead of these generic calculations, we propose the use of an addition chain using point doubling and point addition to generate all necessary points depending only on the value M_{MIN} , or more precisely B_1 . Our implemented version of this chain using the parameter $B_1 = 960$ executes 30 point additions and doubles seven times. It calculates all 27 points and stores two additional temporary points. We recall that the calculation of $R \leftarrow R + Q$ in each iteration (line 32) needs the difference $R - Q$. In the first iteration $R' := R - Q = M_{MIN}DQ_0 - DQ_0 = (M_{MIN} - 1)DQ_0$. We show later that it is sufficient to calculate this point.

Table 6.5 shows the addition chain for $B_1 = 960$ and the resulting $M_{MIN} = 5$. The last five calculations depend on the parameter B_1 as they calculate both $(M_{MIN} - 1) \cdot 210$ and $M_{MIN} \cdot 210$. We implement the addition chain as a hard-coded ALU sequence in the global memory of each ECM core. If a different boundary is used, the sequence can be adjusted easily without any hardware changes.

Algorithm 9 shows the modified implementation of ECM phase 2. The precomputations are split into two parts: Host computations and ECM core computations. The point R' contains the point $R - Q$ for use in the main calculation. In the implementation, we use a single bit to determine which block holds R' and which currently stores R .

Additionally, we modified the instructions sequence of the point addition to write the result synchronously as shown in Figure 6.19. This allows us to calculate $R' = R + Q$ using $R' = R - Q$ without using a temporary storage.

¹⁴see [GKB⁺06], p. 12

Algorithm 9 Implementation of the Standard Continuation Algorithm of phase 2

Input: Composite integer N to be factored, elliptic curve $\mathcal{E}$, $Q_0 = kP_0$ as result of phase 1, B_1 smoothness bound for phase 1, B_2 smoothness bound for phase 2, $B_2 > B_1$, time-memory trade-off parameter $D = 210$.

Output: Factor q of N , $1 < q \leq N$ or FAIL

Precomputations [Host]:

```

1:  $M_{MIN} \leftarrow \left\lfloor \frac{B_1}{210} + 0.5 \right\rfloor$ ,  $M_{MAX} \leftarrow \left\lceil \frac{B_2}{210} - 0.5 \right\rceil$ 
2: pre-calculate 24 elements of  $J_S = \{j \mid 1 \leq j \leq 52, \gcd(j, 210) = 1\}$  on host
3: initialize primetable with 0
4: for each  $m = M_{MIN}$  to  $M_{MAX}$  do
5:   for each  $j \in J_S$  do
6:     if  $210 \cdot m + j$  or  $210 \cdot m - j$  is prime then
7:       prime_table[ $m, j$ ] = 1
8:     end if
9:   end for
10: end for

```

Precomputations [ECM core]:

```

11:  $d \leftarrow 1$ 
12: execute addition chain to  $AC(B_1)$  to generate all points
13: store the points  $jQ_0$  ordered at address 0 to 23
14: store the points  $Q \leftarrow DQ_0$ ,  $R \leftarrow M_{MIN}DQ_0$  and  $R' \leftarrow (M_{MIN} - 1)DQ_0$ 

```

Main computations [ECM core]:

```

15: for each  $m = M_{MIN}$  to  $M_{MAX}$  do
16:   retrieve 24 bits of primetable $m$ 
17:   for  $i$  from 0 to 24 do
18:     if primetable $m$ [ $j$ ] = 1 then
19:        $d \leftarrow d \cdot (R_x(jQ_0)_z - (jQ_0)_x R_z) \{R = (R_x : : R_z)\}$ 
20:     end if
21:   end for
22:   calculate  $R + Q$  using difference  $R' = R - Q$ 
23:    $R' \leftarrow R$ ,  $R \leftarrow$  result  $R + Q$  {No temporary storage necessary in hardware.}
24: end for

```

Final computation [Host]:

```

25:  $q \leftarrow \gcd(d, N)$ 
26: if  $q > 1$  then
27:   return  $q$ 
28: else
29:   return FAIL
30: end if

```

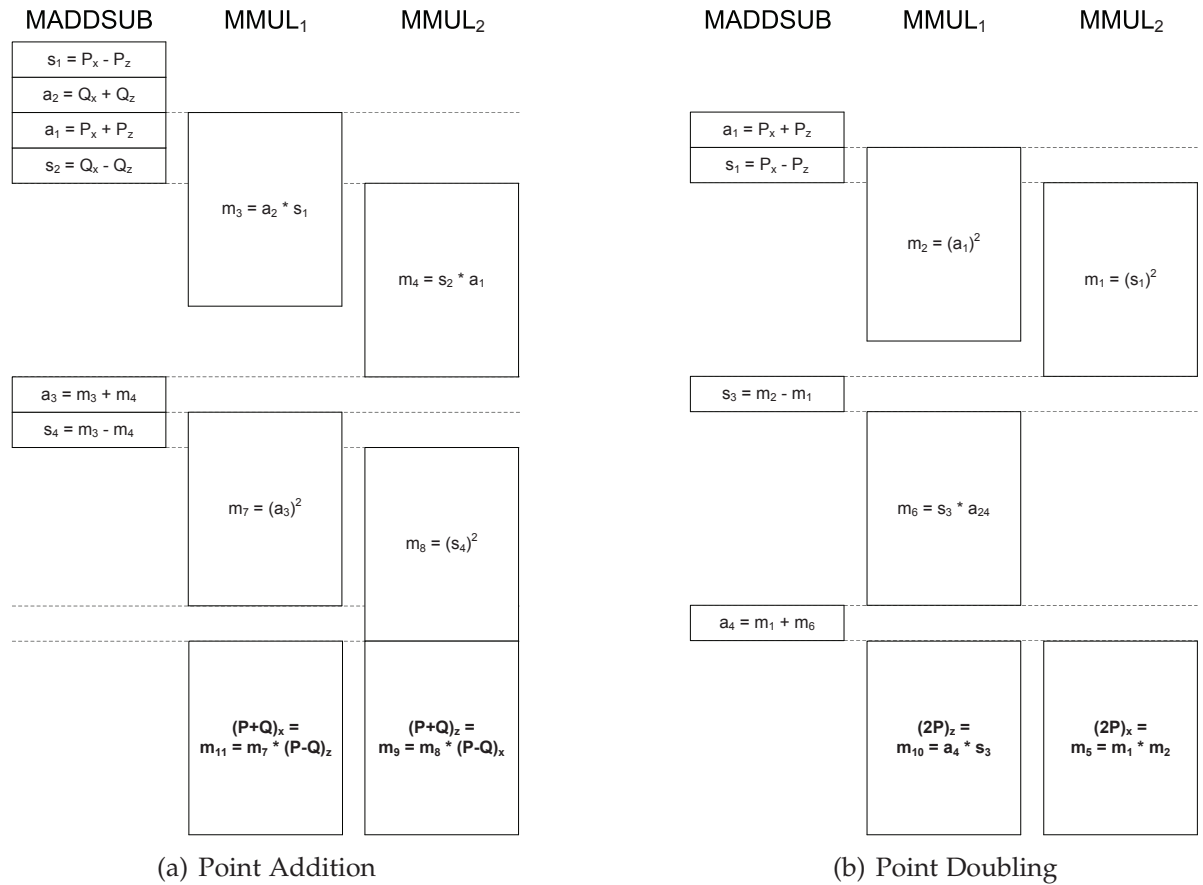


Figure 6.19.: Implementation of point addition and point doubling using two modular multiplication and one modular additions / subtraction unit.

Mode	P	Q	P-Q	Result
Double	1	-	-	2
Add	2	1	1	3
Double	2	-	-	4
Double	3	-	-	6
Add	4	3	1	7
Add	6	3	3	9
Add	7	4	3	11
Add	11	2	9	13
Add	13	4	9	17
Add	13	6	7	19
Add	17	6	11	23
Add	17	13	4	30
Add	23	6	17	29
Add	30	1	29	31
Add	30	7	23	27
Add	30	11	19	41
Add	37	6	31	43
Add	30	17	13	47
Add	47	6	41	53

Mode	P	Q	P-Q	Result
Add	53	6	47	59
Add	31	30	1	61
Add	37	30	7	76
Add	41	30	11	71
Add	67	6	61	73
Add	73	6	67	79
Add	53	30	23	83
Add	59	30	29	89
Add	67	30	37	97
Add	71	30	41	101
Add	73	30	43	103
Add	101	4	97	105
Double	105	-	-	210
Double	210	-	-	420
Double	420	-	-	840
Add	210	105	105	315
Double	315	-	-	630
Add	840	210	630	1050

Table 6.5.: Addition chain for $D = 210$ and $B_1 = 960$. The required points to calculate phase 2 are emphasized.

Main computation

After we improved the pre-computation steps for our design, we have to consider the main computations. We use a state machine to handle request the bits of $primetable_m$ in each round and evaluate the $primetable_m[j]$. This leads to the calculation of

$$d \leftarrow d \cdot (R_x(jQ_0)_z - (jQ_0)_x R_z)$$

in case a pair (m, j) expresses a prime number.

Gaj *et al.* suggests the following calculation (Table 6.6) in round m with the points $R = (R_x : : R_z)$ and $(x_i : : z_i) = j_i Q_0$ for the 24 elements $j_i \in J_S(D)$. He proposed that "after the initial delay of one multiplication, the time required to compute and accumulate any two subsequent values of d_{in} is equal to the time of three multiplications"¹⁵.

¹⁵see [GKB⁺06], p. 13

Adder/Subtractor	Multiplier 1	Multiplier 2
	$m_1 = R_x \cdot z_0$	$m_2 = x_0 \cdot R_z$
$d_0 = m_1 - m_2$ $d_1 = m_3 - m_4$	$m_3 = R_x \cdot z_1$ $d = d \cdot d_0$ $d = d \cdot d_1$	$m_4 = x_1 \cdot R_z$ $m_1 = R_x \cdot z_2$ $m_2 = x_2 \cdot R_z$
$d_2 = m_1 - m_2$ $d_3 = m_3 - m_4$	$m_3 = R_x \cdot z_3$ $d = d \cdot d_2$ $d = d \cdot d_3$	$m_4 = x_3 \cdot R_z$ $m_1 = R_x \cdot z_4$ $m_2 = x_4 \cdot R_z$
...	...	...

Table 6.6.: Accumulation of the partial results $\prod_i (R_x z_i - x_i R_z)$ in phase 2 for a fixed round m , based on [GKB⁺06], p. 13, Table 5

Adder/Subtractor	Multiplier 1	Multiplier 2
	$m_1 = R_x \cdot z_0$	$m_2 = x_0 \cdot R_z$
$d_0 = m_1 - m_2$ $d_1 = m_3 - m_4$	$m_3 = R_x \cdot z_1$ $d = d \cdot d_0$ $d = d \cdot d_1$	$m_4 = x_1 \cdot R_z$ $m_1 = R_x \cdot z_2$ $m_2 = x_2 \cdot R_z$
...	...	...
$d_{n-1} = m_1 - m_2$ $d_n = m_3 - m_4$	$m_3 = R_x \cdot z_3$ $d = d \cdot d_{n-1}$ $d = d \cdot d_n$	$m_4 = x_3 \cdot R_z$

Table 6.7.: Computation of Table 6.6 for an odd number n

We remark that the initial delay exists in every round (and thus $M_{MAX} - M_{MIN} + 1$ times). Additionally, in case of an odd number, the final computation does not utilize the hardware optimally and needs special handling as only one point is available (see Table 6.7).

We can improve the suggested method by changing the evaluation of the row $primetable_m$ and distinguish between 1 and 2 subsequent numbers. Furthermore, we are able to reduce the initial delay to a total of one at the beginning of the algorithm. For description purpose, we use a function `findNextPosition()`. It parses the row and returns the index of the next 1. In case the end is reached, it returns END.

We show how to use this technique to compute the accumulated product using Algorithm 10. We work on the variables r , s and d and delay the computation of $r - s$ to the next iteration. Thus, we require an initialization (line 4) using the first

Algorithm 10 Computation of the d in phase 2

Input: Bit table primetable and Points $R = (R_x :: R_z) = M_{\text{MIN}}DQ_0$, $Q = DQ_0$ and jQ_0 according to algorithm 9

Output: Accumulated result d of Main computation [ECM core], Algorithm 9

```

1:  $m \leftarrow M_{\text{MIN}}, pt \leftarrow \text{primetable}_m$ 
2:  $i \leftarrow \text{findNextPosition}(pt)$  {first prime representation in the table}
3:  $A \leftarrow j_i Q_0$ 
4:  $\mathbf{r} \leftarrow A_z R_x, \mathbf{s} \leftarrow A_x R_z, \mathbf{d} \leftarrow 1$  {initialize  $d, r, s$ }
5: while  $m \leq M_{\text{MAX}}$  do
6:    $i \leftarrow \text{findNextPosition}(pt)$ 
7:   if  $i \neq \text{END}$  then
8:      $A \leftarrow j_i Q_0$  {found prime representation}
9:      $i \leftarrow \text{findNextPosition}(pt)$ 
10:    if  $i \neq \text{END}$  then
11:       $B \leftarrow j_i Q_0$  {found prime representation}
12:       $t \leftarrow A_z R_x, u \leftarrow A_x R_z, d_1 \leftarrow r - s$  {temporary results}
13:       $\tilde{d} \leftarrow d \cdot d_1, \mathbf{r} \leftarrow B_z R_x, d_2 \leftarrow t - u$  {new value of  $r$ }
14:       $\mathbf{d} \leftarrow \tilde{d} \cdot d_2, \mathbf{s} \leftarrow B_x R_z$  {new value of  $d$  and  $s$ }
15:    else
16:       $\mathbf{r} \leftarrow A_z R_x, \mathbf{s} \leftarrow A_x R_z, d_1 \leftarrow r - s$  {new value of  $r, s$ }
17:       $\mathbf{d} \leftarrow d \cdot d_1$ 
18:    end if
19:  end if
20:  if  $i = \text{END}$  then
21:     $m \leftarrow m + 1, pt \leftarrow \text{primetable}_m$  {change the row in primetable}
22:     $R \leftarrow R + Q$ 
23:  end if
24: end while
25:  $d_1 \leftarrow r - s$ 
26:  $\mathbf{d} \leftarrow d \cdot d_1$  {finalize  $d$ }

```

prime we find in the *primetable*. From here on, these variables are updated in each iteration.

Parsing the row m , we try to find two distinct points aQ_0 and bQ_0 and process these subsequent values (lines 11 to 14) using a computation similar to the suggestion. At the end of the iteration, we renew the variables d , r and s .

As soon as we reach the end of *primetable_m* (`findNextPosition()` returns END), we have one of two cases: either we found only aQ_0 , or no point since the last iteration. We cannot use aQ_0 together with the first prime found in *primetable_{m+1}*, because we need to calculate a new point R for the next iteration. Therefore, we update the variables r , s and d using two multiplications (lines 16 and 17). The second case poses no problem and requires no special handling. In both cases we increase m and modify R accordingly to the basic algorithm.

When we reach $m = M_{MAX}$, we still have unused values in r , s and finalize d (lines 25 and 26).

Figure 6.20 shows our state machine handling the second phase of the ECM in our implementation. To optimize the instruction sequence, we use a function to load d and the point R of the current round into the multiplication input buffer using the ALU mode 8. This is executed at the beginning of a new round. During the first run, the mode 9 is used to initialize the values r , s and d . Afterwards *primetable_m* is parsed and the modes 10 and 11 are started accordingly. If necessary, a new memory block is requested and the point addition or - in case of the last round - the finalization is done.

Note that the ALU cannot stop before the new parameters are ready as the state machine fetches the next configuration before the current is started. This ensures that no idle states or side-effects can occur when the number of blocks are changed.

As we already compute the *primetable* on the host, we can analyze the number of iterations we need. Using the parameters $B_1 = 960$, $B_2 = 57000$ and $D = 210$ saves 388 multiplications compared to the suggested implementation by Gaj *et al.*: 265 are saved by the initial delay changes, while the computation using only one point aQ_0 saves additional 123 multiplications.

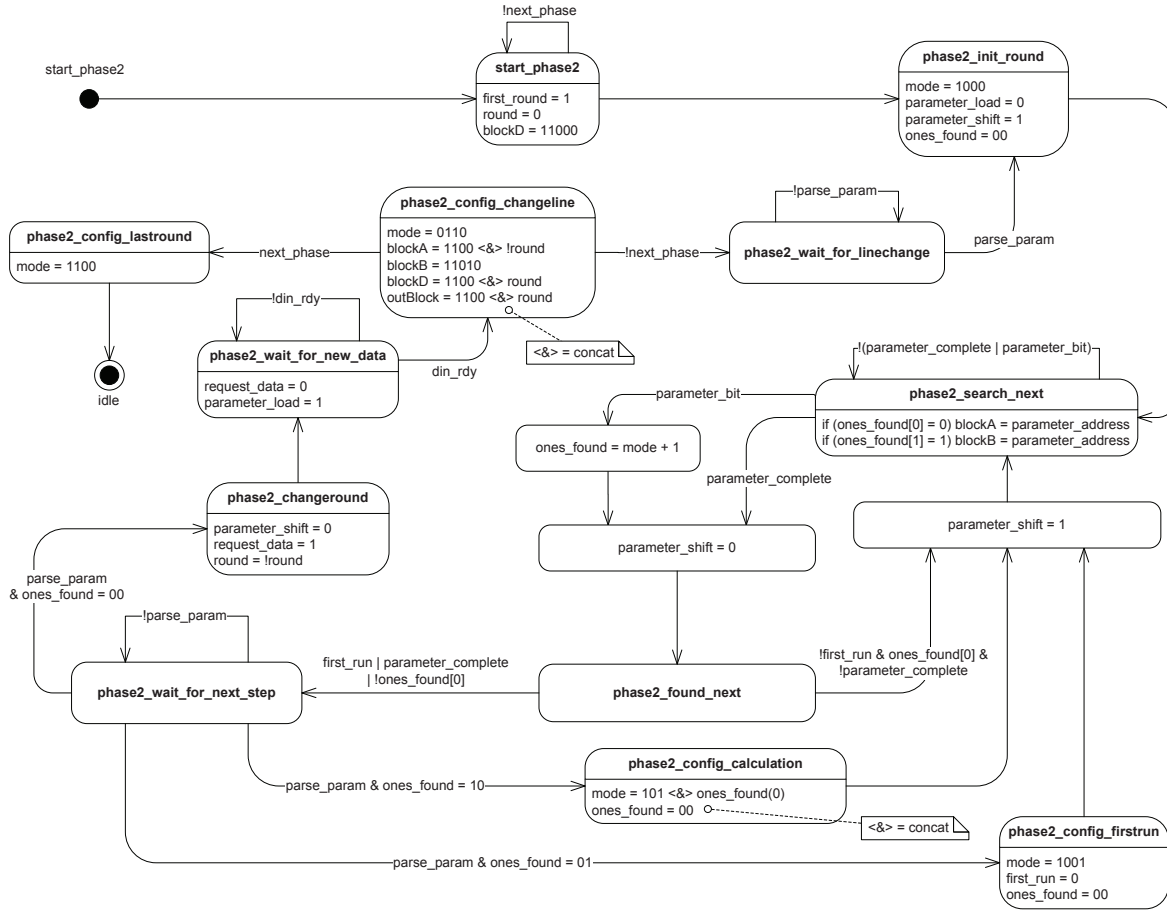


Figure 6.20.: Implementation of Algorithm 10 using a parallel state machine to parse the *primetable*.

7. Results and Comparison

We present the results of our implementation in this chapter: We begin with the arithmetic components and compare them with the implementation by Röpke. Afterwards, we present the ECM system with 24 ECM cores after place and routing and introduce an exact formula to compute the number of clock cycles for every operation used in the arithmetic unit. The last section of this chapter compares our work to related FPGA implementation and the recently published GPU design by Bernstein *et al.*

To synthesize and to place and route our design we use Xilinx Foundation 11. Because we utilize 100% of the available DSP slices for 24 ECM cores, the automatic placement tool by Xilinx did not work: Manual placement was necessary for every ECM system using more than 19 ECM cores. We analyzed the internal data paths and found exactly 2 DSP slices per ECM core, which were not fixed by dependencies. We used iterative, heuristical optimization to place more than 20 ECM cores with a frequency of 200 MHz for the arithmetic unit. Average random placement (fulfilling the design criteria) did not exceed the 160 MHz.

The results we present in this chapter use the ECM parameters $B_1 = 960$, $B_2 = 57000$, $D = 210$ and a scalar k of 1323 bit. These parameters were used in the related work and make the results comparable. As in Chapter 5, the price per FPGA is based on single purchase from the official Xilinx distributor discount.

7.1. Results for the Arithmetic Components

We start with the results for the modular arithmetic component and examine the optimizations on the design from [Röp08]. Both implementations use $b = 10$ blocks of 17 bits and process integers up to 170-bit using a 151-bit modulus, as explained in Section 6.3.3.

We compare the results of our modular multiplication unit with the implementation by Röpke in Table 7.1. Although the change from a parallel to a serialized approach increased the clock cycles per calculation by 91%, the new multiplication unit allows us to improve the number of placeable ECM units by a factor of 3: We reduced the number of digital signal processing elements by 70% and decreased the Slices and LUT usage by 83%. In addition, we were able to increase the post-place

MMUL	Size				Timing		
	DSPs	Flipflops	Slices	LUTs	Cycles	Synth (MHz)	PAR (MHz)
Röpke	10	270	432	517	66	308,613	305,25
This work	3	131	73	83	126	423,012	420,875
Diff	-70%	-51,48%	-83,1%	-83,95%	+90,91%	+37,07%	+37,88%

Table 7.1.: Comparison of the modular multiplication unit to the implementation by Röpke.

and route clock frequency by 37% to the device's maximum as a direct result of this optimization.

As we explained in Section 6.3.3, we implemented two different MADDSUB components. While (a) is a small and highly efficient implementation, the second unit (b) uses dynamic operation modes to control the DSP cores and avoid the shared input per DSP tile. This modification of the original unit was necessary to place more than 19 ECM cores on the FPGA. We recall that this applies to Virtex-4 implementations only, thus (a) can be used i. e., on the new Virtex-5, Virtex-6 and Spartan-6 devices.

MADDSUB	Size				Timing		
	RAM	Flipflops	Slices	LUTs	Cycles	Synth (MHz)	PAR (MHz)
Röpke	1	75	69	121	23	325,31	339,21
(a)	0	106	67	88	23	423,012	421,407
(b)	0	143	104	123	23	409,996	400,000

Table 7.2.: Comparison of both modular addition and subtraction units to the implementation by Röpke.

We present the results for the modular addition and subtraction unit in Table 7.2. We removed the output memory buffer and replaced it by distributed memory, using

LUT and Flipflop resources. In addition, we increased the possible post-place and route clock frequency up to the devices' maximum. While we additionally reduced the area consumption in terms of Slices and LUTs in implementation (a), the extra logic used to change the operation mode increases it slightly in the second version.

7.2. Results for the ECM System

Following the arithmetic components, we present the results for the complete ECM system in this section. Our design was implemented on a Virtex 4 SX35 FPGA and can work using the COPACOBANA framework. Depending on the size of the communication interface, we are able to run up to 24 ECM unit with a clock frequency of 200 MHz for arithmetic operations.

FPGA Type	Virtex-4 SX 35
Number of Cores	24
Arithmetic Clock Frequency	200 MHz
# Cycles (Phase I)	945,746
# Cycles (Phase II)	1,024,029
# Cycles (Phase I + II)	1,969,775
Time (Phase I)	4.73ms
Time (Phase II)	5.12ms
Phase I per Second	5,064
Phase I + II per Second	2,424

Table 7.3.: Implementation results of our ECM system factoring integers up to 151 bit.

The currently implemented instruction sequence factors integers up to 151 bit. We present an overview of the achieved results in Table 7.3: Our design needs 4.72ms to calculate the first phase and 5.12ms for the second phase. Using all 24 parallel ECM cores, we can compute a total of 2,424 ECM computations per second.

Table 7.4 shows the chip utilization of an placed and routed ECM System using only one ECM core. These numbers can be used to quickly compare designs during development and estimate results. We remark that these numbers cannot be scaled to the maximum number of cores and used as results directly, as a high chip utilization changes the routing paths.

	Name	Used / Achieved	Utilization
Size	Flipflops	885 / 30,720	2%
	LUTs	1,007 / 30,720	3%
	Slices	766 / 15,360	4%
	bRAM	7 / 192	3%
	DSPs	8 / 192	4%
Timing	Arithmetic CLK	220 MHz	
	System CLK	100 MHz	
	COPACOBANA Bus	25 MHz	

Table 7.4.: Place & Route utilization of the ECM system with one ECM core

Our ECM system is designed to allow adaptation to smaller or larger bit sizes. It supports bit length of $B = 17 \cdot (b - 1) - 2$ with $b = 5..15$ without notably hardware changes. This concludes a range from 66 to 236 bits. To adjust the bit length, we have to place and route the ECM system with modified generics: This increases or decreases the output buffers of the modular arithmetic units without changing the rest of the architecture. Afterwards, we generate a new instruction sequence and load it into the instruction ROM.

We present the exact formula to determine the number of clock cycles needed to perform the different ECM operations in Table 7.5. They are derived directly from the instruction sequence and represent the actual instruction set. We can approximate the clock cycles by the sequential number of additions and multiplications (neglecting a number of control states): The modular multiplication is critical for the operation timings. The exact formula shows the number of cycles based on the number of blocks b . This formula includes all control states implemented in the instruction set.

While our current implementation uses $b = 10$, Table 7.6 shows the impact on the run-time for larger and smaller numbers. We remark that the only notable adaptation concerns the content of the instruction ROM.

7.3. Comparison

In this section we compare the results of our implementation with the related work. We do not refer to the results of Pelzl *et al.*, as the other publications already im-

Operation	Computation(s)	Formula	Cycles ($b = 10$)
Addition	add	$2b + 3$	23
Multiplication	mul	$b^2 + 2b + 6$	126
$P + Q, 2P$	5·mul+add	$5b^2 + 13b + 58$	688
$P + Q$	3·mul+3·add	$3b^2 + 15b + 51$	501
$2P$	3·mul+add	$3b^2 + 9b + 44$	434
Ph2: init	-	$2b + 4$	24
Ph2: start	mul+6	$b^2 + 2b + 12$	132
Ph2: two	3·mul	$3b^2 + 6b + 32$	392
Ph2: one	2·mul	$2b^2 + 4b + 22$	262
Ph2: end	mul	$b^2 + 3b + 15$	145

Table 7.5.: Formulae to determine the clock cycles per operation depending on the number of 17-bit blocks b . These formulae represent the actual instruction sequence.

proved their implementation. We start with the different FPGA designs and end this section with a brief comparison to the latest results on the elliptic curve method on graphic cards.

We start with a comparison of our design to the implementation by Röpke. We remark that we only compared the first phase, as neither the architecture, nor the arithmetic operations were implemented. Both ECM systems use the same FPGA, the Virtex-4 SX35 with a speed grade of 10, and target the same integers of up to 151 bit. Table 7.7 presents the results: We are able to double the maximum clock frequency and triple the number of ECM cores. As we noticed in the analysis of our operation timings, the modular multiplication dominates the timings: Our implementation needs a factor of 1.91 more clock cycles per multiplication. This factor applies also to the clock cycles per computation of the first phase. As we increased both the number of parallel computations and the clock frequency, we gain an overall improvement factor of 3.15.

With the data from Table 7.6, a fair comparison with designs using different integer bit length is possible without scaling the results. In the following, we exploit our architecture and adapt the maximum bit size by changing the number b of blocks per integer. We use $b = 13$ (factors up to 202 bit integers) for the comparison with Gaj *et al.* and $b = 9$ (factors up to 134 bit integers) to compare our design to de Meulenaer *et al.*

b	max. Bits	Phase 1		Phase 2		Calc / Second	
		cycles	time (ms)	cycles	time (ms)	Phase 1	ECM
5	66	340,916	1.70	367,199	1.84	14,064	6,768
6	83	434,390	2.17	468,413	2.34	11,040	5,304
7	100	541,610	2.71	584,703	2.92	8,856	4,248
8	117	662,576	3.31	716,069	3.58	7,224	3,480
9	134	797,288	3.99	862,511	4.31	6,000	2,880
10	151	945,746	4.73	1,024,029	5.12	5,064	2,424
11	168	1,107,950	5.54	1,200,623	6.00	4,320	2,064
12	185	1,283,900	6.42	1,392,293	6.96	3,720	1,776
13	202	1,473,596	7.37	1,599,039	8.00	3,240	1,560
14	219	1,677,038	8.39	1,820,861	9.10	2,856	1,368
15	236	1,894,226	9.47	2,057,759	10.29	2,520	1,200

Table 7.6.: Results of our implementation for Phase 1 and 2 for different integer bit sizes.

		Röpke	This work	Factor
Design	FPGA type	XC4VSX35-10		
	FPGA price	468 \$		
	Supported Bits	151		
	Max. ECM cores	8	24	3.0
	Max. Frequency	100 MHz	200 MHz	2.0
Timing	Cycles for Addition	23		1.00
	Cycles for Multiplication	66	126	1.91
	Cycles for Phase 1	496,375	945,746	1.91
	Time for Phase 1	4,96ms	4.73ms	0.95
Results	Phase 1 / sec	1,608	5,064	3.15
	Phase 1 / sec per 100 \$	343	1,082	3.15

Table 7.7.: Comparison of our implementation to the design by Röpke.

		Gaj <i>et al.</i>		This work	Factor	
Design	FPGA type	S35000	V4LX200	V4SX35		
	FPGA price	108 \$	7,564 \$	468 \$	4.33	0.06
	Supported Bits	198		202	1.02	
	Max. ECM cores	13	24	24	1.85	1.00
	Max. Frequency	80 MHz	104 MHz	200 MHz	2.5	1.92
Timing	Cycles for Addition	41		29	0.71	
	Cycles for Multiplication	216		201	0.93	
	Cycles for Phase 1	1,666,500		1,473,596	0.88	
	Time for Phase 1	21ms	16ms	7.37ms	0.35	0.46
Results	Phase 1 / sec	624	1,448	3,240	5.19	2.24
	Phase 1 / sec per 100 \$	577	19	692	1.2	36.42

Table 7.8.: Comparison of our results using $b = 13$ with the low-cost Spartan-3 5000 and the high-cost Virtex-4 LX 200 implementation by Gaj *et al.*

Table 7.8 shows the result of the comparison with the implementation by Gaj *et al.* Please note that we compare the first phase only. Even though they present estimates for the second phase, the description omitted at least one calculation of a scalar multiple of Q_0 during the pre-computation steps. This makes a fair comparison not possible. We furthermore selected only two of the five designs: The low-cost Spartan-3 5000 with speed grade 5 as the implementation with the best ratio on cost-effectiveness, and the Virtex-4 LX200 with speed grade 11, because it belongs to the Virtex-4 family and provides a similar architecture as our small Virtex-4 SX.

We regard the low-cost Spartan-3 design first: our design increases the number of maximum ECM cores by a factor of 1.85 from 13 to 24. In addition, we improve the clock frequency from 80 MHz to 200 MHz. As our design is considerably faster and needs only 35% of the run-time to compute the first phase, we gain an overall improvement factor of 5.19 at computations per second. If we consider the low price of the Spartan-3, we still improve their design by a factor 1.2.

The implementation by Gaj *et al.* using the largest Virtex-4 device, the LX200, utilizes the large amount of Slices and can place as many ECM cores as our design. Still, we improve nearly double the frequency and need slightly less clock cycles to compute the first phase. This leads to an overall improvement of our implementation

by a factor of 2.24 concerning the computations per second. If we take the high price of the FPGA into account, we gain a factor of 36.42.

		de Meulenaer <i>et al.</i>	This work	Factor
Design	FPGA type	XCV4SX25-10	XC4VSX35-10	
	FPGA price	298 \$	468 \$	1.57
	Supported Bits	135	134	0.99
	Max. ECM cores	1	24	24.0
	Max. Frequency	220 MHz	200 MHz	0.91
Timing	Cycles for Addition	1	21	21.00
	Cycles for Multiplication	1	105	105.00
	Cycles for Phase 1	13,750	797,288	57.98
	Time for Phase 1	63 μ s	3.99ms	63.33
Results	Phase 1 / sec	16,000	6,000	0.38
	Phase 1 / sec per 100 \$	5,369	1,282	0.24

Table 7.9.: Comparison of our results using $b = 9$ with the fully pipelined, unrolled implementation by de Meulenaer *et al.*

The last FPGA design in this comparison is the implementation by de Meulenaer *et al.* As described in 5, they use the host to pre-compute the point doubling and an extension to the second phase is not straightforward. Thus, this comparison is stated for informational purpose only.

The comparison is presented in Table 7.9. Their design factors integers up to 135 bit and needs 63 μ s for one computation. As our implementation is considerably slower (by a factor of 63.33) per computation and our FPGA has a higher price, our design cannot compete in phase 1. The benefit comes from the different architecture and implemented the Montgomery ladder only.

This section ends with a comparison to the latest implementation of the ECM on graphic cards by Bernstein *et al.* Their implementation is optimized on a high modular multiplication per second ratio and focuses on the first phase. They use a larger smoothness bound $B_1 = 8192$ and target 210 bit numbers. We adjusted our results by using $b = 14$ (factors up to 219 bits) and increased our computation from a skalar k of 1375 bits to one of 11797 bits.

Table 7.10 shows the results of our comparison. The maximum power consumption for the GPU was taken from a test by HardTecs4U on "Power consumption

		Bernstein <i>et al.</i>	This work	Factor
Design	Device Type	GTX 295	XC4VSX35-10	
	Device price	500 \$	468 \$	0.936
	Supported Bits	210	219	1.04
	Max. Power consumption	316.5 W	10 W	0.03
	Max. Frequency	1,242 MHz	200 MHz	0.16
Results	Phase 1 / sec	4,928	312	0.06
	Phase 1 / sec per 100 \$	985	66	0.07
	Phase 1 / sec per W	15	31	2.07

Table 7.10.: Comparison of our results using $b = 14$ and a scalar k of 117979 bits with the latest implementation of the ECM on graphic cards by Bernstein *et al.* in [BCC⁺09]

of current graphic cards”¹⁶. We used the maximum power supplied by the COPA-COBANA per FPGA as a reference, although first tests on our design indicate that it does not require more than 4W.

While the graphic card is about a factor 6.2 faster in terms of the clock frequency, it consumes about 31.6 times more power. This leads to the following results: The graphic card implementation tests 15.8 times as many elliptic curves per second as our implementation. Comparing the number of calculations per Watt, even in the worst case is our design more than twice as efficient as the GTX 295 version. In addition, our architecture is able to run the second phase and not optimized to run the first.

¹⁶see http://ht4u.net/reviews/2009/power_consumption_graphics page 14, visited November, 14th, 2009

8. Conclusion

In this chapter we conclude our results and give an outlook on possible future work.

We presented an implementation of both ECM Phase 1 and Phase 2 and designed an architecture which can run up to 24 parallel ECM cores on Virtex-4 SX35 FPGAs. The design was generically implemented to factor 66 to 236 bit integers without considerable hardware changes. We optimized the algorithm that was suggested by Pelzl *et al.* and Gaj *et al.* to implement the second phase and use an addition chain instead of a generic pre-computation. As the addition chain and instruction sequences are encoded in the memory, we can reuse our design with other ECM parameters B_1 and B_2 : we need to generate new memory data and load it with the bitstream without modifying the design. The optimizations in the main computation of phase 2 additionally reduce the computation time by additional 5% of the computation time.

Our implementation computes 2,424 ECM/s per FPGA running both phases, which scales to a total of 310,272 ECM/s per second on a fully equipped COPA-COBANA cluster using 128 FPGAs. Our design increased the number of ECM/s (per 100\$) by a factor of at least 1.26, compared to all currently published FPGA results implementing or estimating both ECM phases. While we designed our architecture to benefit from the second phase, we achieved an improvement compared to almost all FPGA designs (running only phase 1). Only the fully pipelined implementation by de Meulenaer *et al.* improves our result by a factor of 4.19 at computations/s (per 100\$). We remark that their approach is optimized for running the first phase and makes an implementation of the second phase very difficult. To our knowledge there have been no published results yet.

When we compare our FPGA design to latest implementation on graphic cards, even though they utilize more computational power and outperform our design at computations per second, we achieve a better ratio at calculations per watt by at least factor 2 (worst case). Note that there is no efficient implementation performing both phases of the ECM on GPUs yet.

Future work on the elliptic curve method on FPGAs should evaluate the new Virtex-6 and Spartan-6 devices to benefit from the rapid hardware evolution. Our design will require local optimization of the arithmetic unit, as the number and size of BlockRAMs is different in the new FPGA families. In the course of these optimizations, power saving techniques should be included: We did not use any mechanism to disable unused DSP cores. We can implement this by modifying the

reset instruction behavior. Additionally, if not enough data is provided by the host, the ECM cores could be disabled dynamically to save power.

As the implementation on graphic cards by Bernstein *et al.* uses Edwards curves instead of Montgomery curves, the arithmetic requirements and application on FPGAs should be evaluated.

A. Bibliography

- [BCC⁺09] Daniel J. Bernstein, Hsueh-Chung Chen, Ming-Shing Chen, Chen-Mou Cheng, Chun-Hung Hsiao, Tanja Lange, Zong-Cing Lin, and Bo-Yin Yang. The Billion-Mulmod-Per-Second PC. In *SHARCS'09 Workshop Record*, pages 131–144, 2009.
- [BR96] Stephen Brown and Jonathan Rose. FPGA and CPLD Architectures: A Tutorial. *IEEE Design and Test of Computers*, 13(2):42–57, 1996.
- [Bre86] R. P. Brent. Some integer factorization algorithms using elliptic curves. *Australian Computer Science Communications*, 8(2):149–163, 1986.
- [CFA⁺06] Henri Cohen, Gerhard Frey, Roberto Avanzi, Christophe Doche, Tanja Lange, Kim Nguyen, and Frederik Vercauteren, editors. *Handbook of elliptic and hyperelliptic curve cryptography*. Discrete Mathematics and its Applications (Boca Raton). Chapman & Hall/CRC, Boca Raton, FL, 2006.
- [cKKABSK96] Çetin Kaya Koç, Tolga Acar, and Jr. Burton S. Kaliski. Analyzing and Comparing Montgomery Multiplication Algorithms. *IEEE Micro*, 16(3):26–33, 1996.
- [Dew98] L. Dewaghe. Remarks on the Schoof-Elkies-Atkin Algorithm. *Mathematics of Computation*, 67:67–223, 1998.
- [DH76] W. Diffie and M. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, January 1976.
- [dMGdDQ07] Giacomo de Meulenaer, François Gosset, Gueric Meurice de Dormale, and Jean-Jacques Quisquater. Integer Factorization Based on Elliptic Curve Method: Towards Better Exploitation of Reconfigurable Hardware. In *FCCM*, pages 197–206, 2007.
- [GKB⁺06] Kris Gaj, Soonhak Kwon, Patrick Baier, Paul Kohlbrenner, Hoang Le, Mohammed Khaleeluddin, and Ramakrishna Bachimanchi. Implementing the Elliptic Curve Method of Factoring in Reconfigurable Hardware. In *CHES*, pages 119–133, 2006.
- [GKB⁺08] Kris Gaj, Soonhak Kwon, Patrick Baier, Paul Kohlbrenner, Hoang Le, Mohammed Khaleeluddin, and Ramakrishna Bachimanchi. Area-Time Efficient Implementation of the Elliptic Curve method of Factor-

- ing in Reconfigurable Hardware for Application in the Number Field Sieve. *Draft*, June 2008.
- [Gün09] Tim Erhan Güneysu. *Cryptograpy and Cryptanalysis on Reconfigurable Devices*. Dissertation in Electrical Engineering and Information Sciences, Ruhr-University Bochum, February 2009.
- [GPPS08] Tim Güneysu, Christof Paar, Gerd Pfeiffer, and Manfred Schimmler. Enhancing COPACOBANA for advanced applications in cryptography and cryptanalysis. In *FPL*, pages 675–678. IEEE, 2008.
- [Har04] Laszlo Hars. Long Modular Multiplication for Cryptographic Applications. In Marc Joye and Jean-Jacques Quisquater, editors, *CHES*, volume 3156 of *Lecture Notes in Computer Science*, pages 45–61. Springer, 2004.
- [HMOV04] Darrel Hankerson, Alfred Menezes, and Scott Vanstone. *Guide to elliptic curve cryptography*. Springer Professional Computing. New York, 2004.
- [Knu97] Donald E. Knuth. *Art of Computer Programming, Volume 1: Fundamental Algorithms, Third Edition*. Addison Wesley, 1997.
- [Kob87] Neal Koblitz. Elliptic curve cryptosystems. *Mathematics of Computation*, 48:293–209, 1987.
- [KPP⁺06] S. Kumar, C. Paar, J. Pelzl, G. Pfeiffer, A. Rupp, and M. Schimmler. How to Break DES for 8,980 Euro. In *International Workshop on Special-Purpose Hardware for Attacking Cryptographic Systems — SHARCS’06*, Cologne, Germany, <http://www.sharcs.org>, April 2006.
- [KTR08] Ian Kuon, Russell Tessier, and Jonathan Rose. FPGA Architecture: Survey and Challenges. *Found. Trends Electron. Des. Autom.*, 2(2):135–253, 2008.
- [Len87] H. W. Lenstra, Jr. Factoring Integers with Elliptic Curves. *Annals of Mathematics*, 126:649–673, 1987.
- [Mil86] Victor S. Miller. Use of Elliptic Curves in Cryptography. In *CRYPTO ’85: Advances in Cryptology*, pages 417–426, London, UK, 1986. Springer-Verlag.
- [Mon85] Peter L. Montgomery. Modular Multiplication Without Trial Division. *Mathematics of Computation*, 44(170):519–521, April 1985.
- [Mon87] Peter L. Montgomery. Speeding the Pollard and elliptic curve methods of factorization. *Mathematics of Computation*, 48:243–264, 1987.

- [Mon92] Peter L. Montgomery. Evaluating recurrences of form $x_{m+n} = f(x_m, x_n, x_{m-n})$ via Lucas Chains, 1983, revised January 1992. Available at <ftp.cwi.nl/pub/pmontgom/Lucas.ps.gz>.
- [Oru95] Holger Orup. Simplifying Quotient Determination in High-Radix Modular Multiplication. *IEEE Symposium on Computer Arithmetic*, 0:193, 1995.
- [Pol74] John M. Pollard. Theorems on factorization and primality testing. *Proceedings of the Cambridge Philosophical Society*, 76:521–528, 1974.
- [PSK⁺05] J. Pelzl, M. Simka, T. Kleinjung, J. Franke, C. Priplata, C. Stahlke, M. Drutarovsky, V. Fischer, and C. Paar. Area-time efficient hardware architecture for factoring integers with the elliptic curve method. *IEE Proceedings of Information Security*, 152(1):67–78, 2005.
- [RHSDPK06] Francisco Rodríguez-Henríquez, N. A. Saqib, A. Díaz-Pérez, and Cetin Kaya Koc. *Cryptographic Algorithms on Reconfigurable Hardware (Signals and Communication Technology)*. Springer-Verlag New York, Inc., Secaucus, NJ, USA, 2006.
- [Röp08] C. Röpke. Integer Factorization based on the Elliptic Curve Method using Reconfigurable Hardware. Master’s thesis, Ruhr-Universität Bochum, Department of Electrical Engineering and Information Sciences, 2008.
- [Sch95] Bruce Schneier. *Applied cryptography (2nd ed.): protocols, algorithms, and source code in C*. John Wiley & Sons, Inc., New York, NY, USA, 1995.
- [Sti02] Douglas R. Stinson. *Cryptography: Theory and Practice, Second Edition*. Chapman & Hall/CRC, February 2002.
- [Wil82] Hugh C. Williams. A $p + 1$ method of factoring. *Mathematics of Computation*, 39:225–234, 1982.
- [Wil95] Andrew Wiles. Modular elliptic curves and Fermat’s Last Theorem. *Annals of Mathematics*, 142:443–551, 1995.
- [Xil08a] Xilinx. *Virtex-4 FPGA User Guide*, December 2008. Version 2.6.
- [Xil08b] Xilinx. *XtremeDSP for Virtex-4 FPGAs*, May 2008. Version 2.7.
- [ZD06] Paul Zimmermann and Bruce Dodson. 20 years of ECM. In *Proceedings of the 7th Algorithmic Number Theory Symposium (ANTS VII)*, volume 4076, pages 525–542, 2006.

B. List of Figures

2.1. Basic FPGA layout	6
2.2. CLB in Virtex-4 FPGAs	7
2.3. Distribution of dedicated hardware	8
2.4. Simplified view of a DSP48 Slice	10
2.5. Layout of the modified COPACOBANA	11
3.1. Graph of the elliptic curve $y^2 = x^3 - 3x + 3$ over $\mathbb{R}$	21
3.2. Construction of the point addition: $R = P + Q$, P, Q with $R \in \mathcal{E}(\mathbb{F}_p)$. .	23
5.1. Layout of the ECM Unit	33
6.1. Sequence diagram: host - FPGA	40
6.2. ECM system Timing Diagram	41
6.3. FPGA Layout: Interface and Data Conversion	42
6.4. Layout of the ECM system and ECM core	43
6.5. Module description of Arithmetic-Logic Unit	44
6.6. Components of the arithmetic-logic unit	46
6.7. Description of the Workspace Memory	47
6.8. Layout of one storage block	47
6.9. New instruction ROM design	49
6.10. Input/Output ports of the new module MMUL.	54
6.11. DSP configuration of the modular multiplication unit	54
6.12. Timing diagram of the modular multiplication unit	55
6.13. Parallel vs. sequential: time/area trade-off	56
6.14. Input/Output ports of the new module MADDSUB.	57
6.15. Timing diagram of the modular addition and subtraction unit	58
6.16. State Machine of the control unit	60
6.17. Implementation of the Montgomery Step using P_0 as starting point. .	61
6.18. Implementation of the Montgomery Step using P_0 as starting point. .	63
6.19. Implementation of EC arithmetic	66
6.20. Statemachine of Algorithm 10	71

C. List of Tables

5.1. Results of the related work using identical parameters.	37
6.1. Modes of operation to select with <code>in_mode</code>	45
6.2. List of the different instruction signals and the instruction encoding. .	50
6.3. Storage block input configuration	51
6.4. Instructions per ALU mode	52
6.5. Implemented addition chain for $D = 210$	67
6.6. Accumulation in phase 2	68
6.7. Computation of Table 6.6 for an odd number n	68
7.1. Multiplication unit compared to Röpke	74
7.2. Addition/Subtraction units compared to Röpke	74
7.3. Implementation results of the ECM system	75
7.4. Place & Route utilization of the ECM system with one ECM core . . .	76
7.5. Clock cycle formulae depending on the number of blocks	77
7.6. Timings for different bit sizes	78
7.7. ECM System compared to Röpke	78
7.8. ECM System compared to Gaj <i>et al.</i>	79
7.9. ECM System compared to de Meulenaer <i>et al.</i>	80
7.10. ECM System compared to GPU (Bernstein <i>et al.</i>)	81

D. List of Algorithms

1.	Modular addition/subtraction	17
2.	Modular multiplication	17
3.	Montgomery Reduction $MRed(X)$	18
4.	Elliptic Curve Method - Phase 1	29
5.	Montgomery Ladder Algorithm	30
6.	Elliptic Curve Method - Standard continuation	31
7.	Modular Multiplication with Quotient Pipelining and $d = 0$	53
8.	Standard Continuation Algorithm of phase 2 from [GKB ⁺ 06]	62
9.	Implementation of the Standard Continuation Algorithm of phase 2	65
10.	Computation of the d in phase 2	69